

Практикум на ЭВМ

Подробный разбор трёх заданий

Задание 1. Обращение матрицы методом Гаусса с выбором главного элемента по всей матрице (MPI)

Задание 2. Приближение функций одной переменной:
формула Ньютона, сплайн Эрмита, параболический сплайн (Qt)

Задание 3. Интерполяция функции двух переменных
по формуле Ньютона (тензорные произведения, Qt, 3D)

Как пользоваться этим документом

Документ устроен так, что его можно читать подряд и при этом закрыть все вопросы, которые обычно задают на защите. Для каждого задания есть:

- постановка задачи и точный интерфейс программы (аргументы, клавиши, вывод);
- полная математика с выводом всех формул, а не только их перечисление;
- разбор программы по файлам и функциям, с фрагментами кода и объяснением, зачем нужна каждая строчка;
- таблицы реально полученных численных результатов (все числа в документе получены прогоном именно этих программ, а не взяты из головы);
- список типичных вопросов преподавателя с готовыми ответами.

Соответствие папок в репозитории и глав документа:

Папка	Задание	Глава
<code>task1_yarik</code>	обращение матрицы, MPI	глава 1
<code>task2_yarik</code>	функции одной переменной, Qt	глава 2
<code>task3_yarik</code>	функции двух переменных, Qt, 3D	глава 3

Оглавление

Как пользоваться этим документом	1
1 Задание 1. Обращение матрицы методом Гаусса с выбором главного элемента по всей матрице (MPI)	5
1.1 Постановка задачи	5
1.2 Математика метода	5
1.2.1 Элементарные преобразования строк как умножение на матрицу	5
1.2.2 Идея: приписываем единичную матрицу	6
1.2.3 Прямой и обратный ход	6
1.2.4 Сколько это стоит	6
1.2.5 Зачем выбирать главный элемент	7
1.2.6 Главная тонкость: перестановка столбцов	7
1.2.7 Критерий вырожденности	7
1.3 Параллельная реализация	8
1.3.1 Как распределены данные	8
1.3.2 Что пересылается на шаге прямого хода	8
1.3.3 Обратный ход и расстановка строк	8
1.3.4 Невязки без дополнительной памяти	9
1.4 Разбор программы	9
1.4.1 Файлы	9
1.4.2 <code>main.c</code>	9
1.4.3 <code>input.c</code>	10
1.4.4 <code>output.c</code>	10
1.4.5 <code>gauss_reverse_MPI.c</code> : сам метод	10
1.5 Проверки: что реально получилось	11
1.6 Вопросы, которые обычно задают	12
2 Задание 2. Приближение функций одной переменной	13
2.1 Постановка	13
2.2 Метод 1. Интерполяционная формула Ньютона	13
2.2.1 Существование и единственность	13
2.2.2 Разделённые разности	13
2.2.3 Вывод формулы Ньютона	14
2.2.4 Вычисление коэффициентов «на месте»	14
2.2.5 Вычисление значения: схема Горнера	14
2.2.6 Погрешность и эффект Рунге	15
2.3 Метод 2. Кубические многочлены Эрмита, естественные граничные условия (задача 13)	15
2.3.1 Постановка	15
2.3.2 Вывод коэффициентов	15
2.3.3 Откуда берутся d_i во внутренних узлах	15
2.3.4 Естественные граничные условия: вывод d_1 и d_n	16

2.3.5	Точность	16
2.3.6	Вычисление значения	17
2.4	Метод 3. Параболические сплайны, граничные условия экстраполяцией (задача 46)	17
2.4.1	Почему точки склейки берутся между узлами	17
2.4.2	Параметризация через значения в точках склейки	17
2.4.3	Уравнения непрерывности производной	18
2.4.4	Проверка на равномерной сетке	18
2.4.5	Граничные условия: экстраполяция	18
2.4.6	Метод прогонки	19
2.4.7	Вычисление значения	19
2.5	Графическая часть (Qt)	19
2.5.1	Из чего состоит программа	19
2.5.2	Преобразование координат	20
2.5.3	Подсчёт масштаба	20
2.5.4	Рисование графика	20
2.5.5	Память и время при больших n	20
2.5.6	Что происходит при нажатии клавиши	20
2.5.7	Как это выглядит	21
2.6	Проверки: что реально получилось	23
2.7	Вопросы, которые обычно задают	23
3	Задание 3. Интерполяция функции двух переменных по формуле Ньютона	25
3.1	Постановка	25
3.2	Математика: тензорные произведения	25
3.2.1	Базис	25
3.2.2	Почему это действительно интерполяция	26
3.2.3	Алгоритм: два прохода по таблице	26
3.2.4	Вычисление значения: двойная схема Горнера	27
3.3	Выбор узлов: Чебышёв и порядок Лежа	27
3.3.1	Почему не равномерные узлы	27
3.3.2	Почему важен порядок узлов (тонкое место)	27
3.4	Трёхмерная графика без OpenGL	28
3.4.1	Что рисуется	28
3.4.2	Нормировка и проекция	28
3.4.3	Видимость: алгоритм художника	28
3.5	Разбор программы	29
3.6	Как это выглядит	29
3.7	Проверки: что реально получилось	32
3.8	Вопросы, которые обычно задают	32
A	Сборка: какие библиотеки нужны	34
A.1	Задание 1 (MPI)	34
A.2	Задания 2 и 3 (Qt)	34
B	Шпаргалка по клавишам	35
B.1	Задание 2	35
B.2	Задание 3	35
C	Частые ошибки и как их чинить	36

D	Полные тексты файлов с методами	37
D.1	Задание 1: <code>gauss_reverse_MPI.c</code>	37
D.2	Задание 1: <code>input.c</code>	41
D.3	Задание 1: <code>output.c</code>	42
D.4	Задание 1: <code>main.c</code>	43
D.5	Задание 2: <code>newton_interpolation.cpp</code>	45
D.6	Задание 2: <code>hermite_natural.cpp</code>	46
D.7	Задание 2: <code>parabolic_extrapolation.cpp</code>	47
D.8	Задание 3: <code>newton2d.cpp</code>	50

Глава 1

Задание 1. Обращение матрицы методом Гаусса с выбором главного элемента по всей матрице (MPI)

1.1 Постановка задачи

Дана вещественная матрица A размера $n \times n$. Требуется найти обратную матрицу A^{-1} , то есть такую матрицу X , что

$$AX = XA = E,$$

где E — единичная матрица. Вычисления должны быть распараллелены на p процессов с помощью MPI; матрица целиком ни в один процесс не помещается, поэтому каждый процесс хранит только свою часть строк. Дополнительно программа должна уметь строить матрицу по формулам, читать её из файла, печатать фрагмент матрицы и считать невязки.

Интерфейс программы.

```
mpirun -np p ./a.out n r s [filename]
```

- n — размер матрицы;
- r — сколько строк и столбцов печатать (чтобы не залить экран);
- s — номер формулы, по которой строится матрица; $s = 0$ — читать из файла;
- **filename** — имя файла, нужен только при $s = 0$.

Формулы (нумерация i, j с единицы):

$$s = 1: a_{ij} = n - \max(i, j), \quad s = 2: a_{ij} = \max(i, j), \quad s = 3: a_{ij} = |i - j|, \quad s = 4: a_{ij} = \frac{1}{i + j - 1}.$$

Последняя — матрица Гильберта, знаменитая своей плохой обусловленностью; она специально нужна, чтобы увидеть, как метод ведёт себя на почти вырожденной матрице.

Программа печатает строку вида

```
./a.out : Task = 14 Res1 = ... Res2 = ... T1 = ... T2 = ... S = ... N = ...
```

где $\text{Res1} = \|AA^{-1} - E\|$, $\text{Res2} = \|A^{-1}A - E\|$, T_1 — время обращения, T_2 — время подсчёта невязок. Если матрица вырождена, печатается $\text{Res1} = \text{Res2} = -1$.

1.2 Математика метода

1.2.1 Элементарные преобразования строк как умножение на матрицу

Всё, что делает метод Гаусса со строками, есть умножение слева на невырожденную матрицу:

- умножение строки k на число $c \neq 0$ — умножение на диагональную матрицу D , у которой на месте (k, k) стоит c ;

- прибавление к строке i строки k , умноженной на c — умножение на матрицу $E + ce_{ik}$, где e_{ik} — матричная единица;
- перестановка строк i и k — умножение на матрицу перестановки P_{ik} .

Поэтому вся последовательность преобразований строк — это одна матрица M , равная произведению всех использованных элементарных матриц. Аналогично перестановка столбцов — это умножение *справа* на матрицу перестановки.

1.2.2 Идея: приписываем единичную матрицу

Возьмём блочную матрицу $[A \mid E]$ и будем делать со строками такие преобразования, чтобы левый блок стал единичным. Пусть все преобразования собрались в матрицу M :

$$M[A \mid E] = [MA \mid M] = [E \mid M].$$

Из $MA = E$ следует $M = A^{-1}$. То есть справа сама собой получается обратная матрица. В программе левый блок — это массив `local_a`, правый — `local_x`, и они хранятся как две отдельные матрицы (сцеплять их в одну матрицу $n \times 2n$ не нужно).

1.2.3 Прямой и обратный ход

Прямой ход. На шаге $k = 0, \dots, n-1$:

1. выбирается главный элемент и ставится в позицию (k, k) (см. ниже);
2. строка k делится на a_{kk} , так что на диагонали получается единица;
3. из всех строк $i > k$ вычитается строка k , умноженная на a_{ik} ; при этом в столбце k ниже диагонали появляются нули.

После прямого хода матрица A превратилась в верхнюю треугольную с единицами на диагонали.

Обратный ход. Теперь $k = n-1, \dots, 1$: из всех строк $i < k$ вычитается строка k , умноженная на a_{ik} ; зануляется то, что осталось *над* диагональю. После этого левый блок — единичная матрица, а правый — ответ.

Замечание 1.1. Именно так выглядит метод Гаусса (две отдельные фазы). В методе Жордана нули над диагональю делаются сразу на шаге k прямого хода, отдельного обратного хода нет. Арифметическая сложность у них одинакова с точностью до константы, но у схемы Гаусса на каждом шаге прямого хода меньше работы, а обратный ход дешевле, потому что матрицу A на нём трогать уже не нужно: интересуется только правый блок. В программе это видно — в цикле обратного хода обновляется только `local_x`.

1.2.4 Сколько это стоит

Прямой ход: на шаге k надо обновить $\approx (n-k)$ строк левого блока длины $(n-k)$ и n столбцов правого блока, то есть $\approx (n-k)((n-k) + n)$ умножений и столько же сложений. Суммируя по k :

$$\sum_{k=0}^{n-1} (n-k)((n-k) + n) = \sum_{m=1}^n m(m+n) = \frac{n^3}{3} + \frac{n^3}{2} + O(n^2) = \frac{5}{6}n^3 + O(n^2).$$

Обратный ход: на шаге k обновляется k строк правого блока длины n , итого $\sum_k kn = n^3/2$. Всего порядка $\frac{4}{3}n^3$ умножений и столько же сложений, то есть $\Theta(n^3)$ — как и должно быть для обращения матрицы.

Памяти нужно $2n^2 + O(n)$ чисел: две матрицы (исходная и ответ) плюс буфер на две строки и массив перестановки. Это существенно: в программе нет копии исходной матрицы, поэтому для подсчёта невязки она читается (или строится по формуле) заново.

1.2.5 Зачем выбирать главный элемент

Если на диагонали окажется нуль, делить нельзя. Но проблема не только в точном нуле: если a_{kk} мал по модулю, то множители $c = a_{ik}/a_{kk}$ огромны, и при вычитании происходит катастрофическая потеря точности — значащие цифры исходных данных «съедаются» большими числами. Простой пример в арифметике с тремя значащими цифрами:

$$\begin{pmatrix} 10^{-4} & 1 \\ 1 & 1 \end{pmatrix}, \quad c = \frac{1}{10^{-4}} = 10^4, \quad 1 - 10^4 \cdot 1 = -9999 \approx -10^4,$$

и информация о единице в правом нижнем углу теряется полностью. Если же сначала переставить строки, все множители будут по модулю ≤ 1 и ничего не потеряется.

Существует классическая оценка: при выборе главного элемента по столбцу элементы матрицы могут вырасти в 2^{n-1} раз (теоретический худший случай), а при выборе *по всей матрице* рост ограничен гораздо более медленной функцией $\sim n^{\frac{1}{2}} n^{\frac{\ln n}{4}}$. Поэтому полный выбор главного элемента — самая устойчивая (и самая дорогая по числу сравнений) стратегия. В задании требуется именно она.

1.2.6 Главная тонкость: перестановка столбцов

При полном выборе главный элемент — это максимальный по модулю элемент *всей* оставшейся подматрицы (строки и столбцы с номерами $\geq k$). Чтобы поставить его в угол (k, k) , нужно переставить и строки, и столбцы.

Перестановка строк $[A \mid E]$ — это умножение слева, её можно делать свободно, если одновременно переставлять строки обоих блоков. А вот перестановка столбцов — умножение справа, и она меняет ответ. Разберёмся, как именно.

Пусть P — итоговая матрица всех преобразований строк (включая перестановки), Q — матрица всех перестановок столбцов, применённых к A . К концу работы алгоритма левый блок стал единичным:

$$P A Q = E \implies P = (A Q)^{-1} = Q^{-1} A^{-1} = Q^T A^{-1},$$

а в правом блоке лежит как раз $P \cdot E = P$. Значит, в правом блоке лежит не A^{-1} , а $Q^T A^{-1}$ — обратная матрица с *переставленными строками*. Чтобы получить ответ, надо домножить слева на Q :

$$A^{-1} = Q X, \quad X \text{ — то, что получилось в правом блоке.}$$

Умножение на матрицу перестановки слева — это просто перестановка строк. Программа хранит перестановку столбцов в массиве `col` и в самом конце расставляет строки ответа по местам.

1.2.7 Критерий вырожденности

Матрица объявляется вырожденной, если максимальный по модулю элемент оставшейся подматрицы не превосходит порога

$$\varepsilon = 10^{-14} \|A\|, \quad \|A\| = \max_j \sum_i |a_{ij}|$$

(норма — максимум по столбцам суммы модулей). Абсолютный порог здесь не годится: если все элементы матрицы порядка 10^{-8} , то «маленький» элемент 10^{-10} совершенно нормален, а если элементы порядка 10^8 , то элемент 1 уже фактически нуль. Поэтому порог делается относительным.

1.3 Параллельная реализация

1.3.1 Как распределены данные

Используется *циклическое* распределение строк: глобальная строка g лежит на процессе $g \bmod p$ в локальной строке g/p (целочисленное деление). Процесс с номером **rank** хранит

$$\text{local_n} = \left\lfloor \frac{n}{p} \right\rfloor + \begin{cases} 1, & \text{rank} < n \bmod p, \\ 0, & \text{иначе} \end{cases}$$

строк, каждая длины n .

Почему циклическое, а не блочное (первые n/p строк первому процессу и так далее)? Потому что на шаге k прямого хода работа идёт только со строками $i > k$: при блочном распределении процессы «выключались» бы по очереди, и к концу работал бы только последний. При циклическом распределении оставшиеся строки всегда равномерно размазаны по всем процессам, и загрузка остаётся ровной до последнего шага.

1.3.2 Что пересылается на шаге прямого хода

1. **Поиск главного элемента.** Каждый процесс ищет максимум по своим строкам с номерами $\geq k$ и столбцам $\geq k$, запоминая вместе со значением линейный индекс $gi \cdot n + j$. Глобальный максимум вместе с его местоположением получается одним вызовом `MPI_Allreduce(..., MPI_DOUBLE_INT, MPI_MAXLOC, ...)`. Операция `MPI_MAXLOC` как раз и придумана для «максимум плюс его номер»: она сравнивает поля `value`, а при равенстве берёт меньший `index`, поэтому результат одинаков на всех процессах и не зависит от числа процессов. Это важно: иначе разные процессы могли бы выбрать разные главные элементы и алгоритм бы рассыпался.
2. **Перестановки.** Строки k и p_i меняются местами в обеих матрицах. Если обе строки лежат на одном процессе — он меняет их локально; если на разных — два процесса обмениваются строками одним вызовом `MPI_Sendrecv_replace` (он безопасен: не может возникнуть взаимной блокировки, и не нужен дополнительный буфер). Перестановка столбцов делается всеми процессами одновременно в своих строках и обмена не требует вовсе.
3. **Рассылка ведущей строки.** Процесс-хозяин строки k делит её на главный элемент (и в A , и в X), складывает обе половины в один буфер длины $2n$ и рассылает его всем одним `MPI_Bcast`. Одна рассылка вместо двух — это экономия на латентности: время передачи сообщения примерно $\alpha + \beta L$, где α — задержка старта, β — время на число, L — длина; при n шагах экономится $n\alpha$.
4. **Вычитание.** Каждый процесс независимо вычитает ведущую строку из своих строк с номерами $> k$. Пересылок здесь нет — это и есть та часть, ради которой всё затевалось.

Итого за все n шагов пересылается $O(n^2)$ чисел, а арифметики делается $O(n^3)$: при больших n счёт заведомо перевешивает обмены, и параллельная версия имеет смысл.

1.3.3 Обратный ход и расстановка строк

Обратный ход устроен так же, но рассылается только строка правого блока (матрица A уже не нужна), а вычитание идёт из строк с номерами $< k$.

Расстановка строк по местам делается по массиву `col`, который одинаков на всех процессах (все делают одни и те же перестановки столбцов). Цикл

```
for (i = 0; i < n; ++i) {
    while (col[i] != i) {
        j = col[i];
        swap_rows_mpi(local_x, n, i, j, rank, size);
        col[i] = col[j];
        col[j] = j;
    }
}
```

— это стандартный способ применить перестановку «на месте», разложив её в произведение циклов: каждая строка переставляется не более одного раза, всего не более $n - 1$ обменов. Поскольку `col` одинаков всюду, все процессы выполняют одну и ту же последовательность обменов и не рассогласуются.

1.3.4 Невязки без дополнительной памяти

Нужно посчитать $\|AB - E\|$, где обе матрицы распределены по строкам. Процессу для вычисления элемента $(AB)_{it}$ нужна вся t -я колонка B , а у него есть только её куски. Решение — кольцевой обмен: куски колонки гоняются по кольцу процессов $\text{rank} \rightarrow \text{rank} - 1$, и за p пересылок каждый процесс увидит всю колонку и вернёт свой кусок на место. Пересылка оформлена через производный тип `MPI_Type_vector`, описывающий «столбец локальной матрицы»: $n/p + 1$ элементов с шагом n . Дополнительной памяти при этом не требуется вовсе — используется тот же буфер на $2n$ чисел.

Обе невязки считаются одной и той же функцией: `Res1` — это $r(A, X)$, `Res2` — это $r(X, A)$. Считать обе полезно: например, если в программе перепутан порядок перестановок, одна из невязок остаётся маленькой, а вторая — нет.

1.4 Разбор программы

1.4.1 Файлы

Файл	Что внутри
<code>main.c</code>	разбор аргументов, выделение памяти, вызовы, печать отчёта
<code>input.c/.h</code>	чтение матрицы из файла и построение по формулам
<code>output.c/.h</code>	печать фрагмента распределённой матрицы
<code>gauss_reverse_MPI.c/.h</code>	сам метод и подсчёт невязок
<code>Makefile</code>	сборка

1.4.2 `main.c`

Порядок действий: `MPI_Init` → разбор аргументов → выделение памяти → ввод матрицы → печать фрагмента → замер времени и вызов `gauss_reverse_mpi` → печать ответа → повторное чтение A → невязки → отчёт → `MPI_Finalize`.

Ключевые места:

```
local_a = (double*)malloc((n/size+1)*n*sizeof(double));
local_x = (double*)malloc((n/size+1)*n*sizeof(double));
buffer = (double*)malloc(2*n*sizeof(double));
col = (int*)malloc(n*sizeof(int));
```

Размер $(n/size+1)*n$ — это «своё число строк с запасом на единицу», чтобы не писать отдельную формулу для процессов, которым досталась лишняя строка. Буфер ровно $2n$: в первой половине ведущая строка матрицы A , во второй — матрицы X .

```
MPI_Barrier(MPI_COMM_WORLD);
t1 = MPI_Wtime();
flag = gauss_reverse_mpi(n, local_a, local_x, rank, size, buffer, col);
MPI_Barrier(MPI_COMM_WORLD);
t1 = MPI_Wtime() - t1;
```

Барьеры до и после нужны для честного замера: без первого барьера в таймер попадёт время, за которое процессы «доползают» до начала счёта, без второго — время будет измерено по самому быстрому процессу, а не по самому медленному.

```
/* no memory for a copy of A, so we read it again */
input_mpi(s, filename, n, local_a, buffer, rank, size);
```

Исходная матрица испорчена прямым ходом. Копию держать нельзя (ограничение $2n^2$ памяти), поэтому перед подсчётом невязки матрица восстанавливается тем же способом, каким была получена.

1.4.3 input.c

Две ветки. При $s \neq 0$ каждый процесс сам вычисляет свои строки по формуле — пересылок нет вообще:

```
for (i = 0; i < local_n; ++i)
    for (j = 0; j < n; ++j)
        local_a[i*n+j] = formula_mpi(s, n, i*size+rank, j);
```

Выражение $i*size+rank$ — это перевод локального номера строки в глобальный (обратная операция к $g*size, g/size$).

При $s = 0$ файл читает только процесс 0 (одновременное чтение одного файла многими процессами — источник неприятностей) и сразу отправляет очередную строку её хозяину `MPI_Send`’ом, а хозяин принимает её `MPI_Recv`’ом прямо в нужное место своего массива. Ошибки открытия и формата рассылаются всем через `MPI_Bcast` флага — иначе часть процессов завершилась бы, а часть повисла бы в ожидании.

1.4.4 output.c

Печатается не более r строк и r столбцов. Строка, лежащая не на нулевом процессе, пересылается на нулевой, и печать всегда делает только он — иначе строки от разных процессов перемешались бы в терминале.

1.4.5 gauss_reverse_MPI.c: сам метод

Норма и порог.

```
for (i = 0; i < local_n; ++i)
    for (j = 0; j < n; ++j)
        row_a[j] += fabs(local_a[i*n+j]);
MPI_Allreduce(row_a, row_x, n, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
```

Каждый процесс суммирует модули по своим строкам, получая частичные суммы по столбцам; `MPI_Allreduce` с операцией `MPI_SUM` складывает их покомпонентно, после чего максимум по j даёт норму. Здесь же видно, зачем буфер разбит на две половины `row_a` и `row_x`: у `MPI_Allreduce` вход и выход не должны совпадать.

Инициализация $X = E$.

```
local_x[i*n+j] = (i*size+rank == j ? 1. : 0.);
```

Единица ставится там, где глобальный номер строки совпал с номером столбца.

Поиск главного элемента.

```
local_max.value = -1; local_max.index = 0;
for (i = 0; i < local_n; ++i) {
    gi = i*size + rank;
    if (gi < k) continue;
    for (j = k; j < n; ++j) {
        tmp = fabs(local_a[i*n+j]);
        if (tmp > local_max.value) { local_max.value = tmp; local_max.index = gi*n + j; }
    }
}
MPI_Allreduce(&local_max, &global_max, 1, MPI_DOUBLE_INT, MPI_MAXLOC, MPI_COMM_WORLD);
if (global_max.value <= eps) return -1;
pi = global_max.index/n; pj = global_max.index - pi*n;
```

Координаты (i, j) упакованы в одно целое $gi \cdot n + j$ именно потому, что `MPI_MAXLOC` умеет нести с максимумом ровно одно целое число. Обратная распаковка — деление с остатком.

Перестановки и нормировка. Разобраны выше. Обратите внимание: строки переставляются в *обеих* матрицах, столбцы — только в A (в X столбцы переставлять нельзя, они отвечают за столбцы единичной матрицы).

Вычитание.

```
for (i = 0; i < local_n; ++i) {
    gi = i*size + rank;
    if (gi <= k) continue;
    c = local_a[i*n+k];
    if (c == 0.) continue;
    for (j = k; j < n; ++j) local_a[i*n+j] -= c*row_a[j];
    for (j = 0; j < n; ++j) local_x[i*n+j] -= c*row_x[j];
}
```

Левый блок обновляется только с $j \geq k$: левее уже нули, их пересчитывать незначит. Правый блок — целиком, потому что в нём нулевой структуры нет. Проверка $c == 0.$ пропускает строки, где вычитать нечего (типичная ситуация для разреженных формул вроде $s = 3$).

Обратный ход. Рассылается только строка X , и вычитание идёт из строк выше. Матрица A не обновляется — в этом и состоит экономия.

1.5 Проверки: что реально получилось

Все числа ниже получены прогоном программы (MPICH, 2 физических ядра).

1. Матрица из файла a.txt.

```
mpirun -np 4 ./a.out 4 4 0 a.txt
-4.000e-01  5.000e-01  0.000e+00  1.000e-01
 5.000e-01 -1.000e+00  5.000e-01 -0.000e+00
 5.551e-17  5.000e-01 -1.000e+00  5.000e-01
 1.000e-01  1.110e-16  5.000e-01 -4.000e-01
Task = 14 Res1 = 3.330669e-16 Res2 = 6.938894e-16 T1 = 0.04 T2 = 0.10 S = 0 N = 4
```

Ответ совпадает с аналитическим, невязки — на уровне машинной точности $\varepsilon_{\text{маш}} \approx 2.2 \cdot 10^{-16}$.

2. Формулы, $n = 500$, разное число процессов.

p	$s = 1$	$s = 2$	$s = 3$
1	$3.5 \cdot 10^{-10}$	$8.3 \cdot 10^{-10}$	$2.3 \cdot 10^{-10}$
2	$6.2 \cdot 10^{-10}$	$8.6 \cdot 10^{-10}$	$2.2 \cdot 10^{-10}$
4	$5.4 \cdot 10^{-10}$	$8.1 \cdot 10^{-10}$	$2.3 \cdot 10^{-10}$

Невязка *не зависит* от числа процессов по порядку величины — признак того, что распараллеливание не испортило численную схему. Абсолютная величина $\sim 10^{-10}$ для $n = 500$ объясняется просто: невязка растёт примерно как $n \text{cond}(A) \varepsilon_{\text{маш}}$.

3. Ускорение. $n = 1000$, $s = 1$: на одном процессе $T_1 = 2,49$ с, на двух $T_1 = 1,67$ с, то есть ускорение 1,49 на двух ядрах (эффективность 75%). Запускать больше процессов, чем ядер, бессмысленно: на этой машине при $p = 4$ время выросло до 9–10 с — процессы дерутся за два ядра, а обмены остаются.

4. Матрица Гильберта ($s = 4$), $p = 2$.

n	Res1	Res2
5	$9,1 \cdot 10^{-12}$	$2,6 \cdot 10^{-11}$
8	$3,0 \cdot 10^{-7}$	$9,7 \cdot 10^{-7}$
10	$3,1 \cdot 10^{-4}$	$8,8 \cdot 10^{-4}$
12	-1 (объявлена вырожденной)	-1

Это не ошибка программы, а свойство задачи: число обусловленности матрицы Гильберта растёт примерно как $e^{3,5n}$, и уже при $n = 12$ оно превышает 10^{16} — в арифметике `double` такая матрица неотличима от вырожденной. Программа честно сообщает об этом, а не выдаёт мусор.

5. Вырожденная матрица. Для $\begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 3 & 6 & 9 \end{pmatrix}$ (все строки пропорциональны) печатается Res1 = Res2 = -1, обращение не выполняется.

1.6 Вопросы, которые обычно задают

Почему Гаусс, а не Жордан? У Гаусса две фазы: прямой ход делает нули под диагональю, обратный — над. На обратном ходе матрицу A трогать уже не нужно, поэтому он вдвое дешевле соответствующей части Жордана.

Почему выбор по всей матрице дороже? На шаге k просматривается вся подматрица $(n-k)^2$ элементов вместо столбца длины $(n-k)$. Суммарно это $\Theta(n^3)$ сравнений против $\Theta(n^2)$. Зато гарантированно все множители $|c| \leq 1$ и рост элементов минимален.

Что будет, если не переставлять строки ответа в конце? Получится $Q^T A^{-1}$ — правильные строки, но в неправильном порядке. Невязка $\|AX - E\|$ при этом будет большой, так что ошибка заметна сразу.

Почему MPI_MAXLOC, а не MPI_MAX с отдельным поиском? Потому что нужен не только максимум, но и его расположение, причём одинаковое у всех процессов. MPI_MAXLOC решает это за один коллективный вызов и детерминированно разрешает ничьи.

Почему циклическое распределение строк? Чтобы загрузка оставалась равномерной: на шаге k работают только строки ниже k -й, и при блочном распределении процессы отключались бы по очереди.

Почему нет взаимных блокировок при обмене строками? MPI_Sendrecv_replace по стандарту выполняет отправку и приём так, чтобы не зависеть от размера внутренних буферов; пара процессов вызывает его симметрично, остальные в этот момент ничего не делают.

Почему невязка $\sim 10^{-10}$, а не 10^{-16} ? Потому что невязка масштабируется числом обусловленности и размером: $\|AX - E\| \lesssim cn \operatorname{cond}(A) \varepsilon_{\text{маш}}$. Для $n = 500$ и умеренно обусловленной матрицы это как раз 10^{-10} .

Глава 2

Задание 2. Приближение функций одной переменной

2.1 Постановка

На отрезке $[a, b]$ задана функция f из списка

$$\begin{aligned} k = 0: f = 1, \quad k = 1: f = x, \quad k = 2: f = x^2, \quad k = 3: f = x^3, \\ k = 4: f = x^4, \quad k = 5: f = e^x, \quad k = 6: f = \frac{1}{25x^2 + 1}. \end{aligned}$$

По её значениям в n узлах нужно построить приближения тремя методами и показать в окне Qt графики функции, приближений и погрешностей. В моём варианте реализованы задачи

- **№1** — интерполяционная формула Ньютона (многочленный метод);
- **№13** — кусочная интерполяция кубическими многочленами Эрмита с определением недостающих граничных условий из естественных граничных условий;
- **№46** — интерполяция параболическими сплайнами с определением недостающих граничных условий при помощи экстраполяции в приграничных узлах.

Программа запускается как `./graph a b n k` и управляется клавишами 0–7 (сменить функцию, сменить набор графиков, масштаб, число узлов, возмущение). Сетка узлов — равномерная: $x_i = a + i h$, $h = (b - a)/(n - 1)$, $i = 0, \dots, n - 1$.

2.2 Метод 1. Интерполяционная формула Ньютона

2.2.1 Существование и единственность

Утверждение 2.1. Для любых попарно различных узлов $x_1, \dots, x_n$ и любых значений $f_1, \dots, f_n$ существует ровно один многочлен P степени $\leq n - 1$ такой, что $P(x_i) = f_i$ для всех i .

Доказательство. Единственность: если P_1, P_2 — два таких многочлена, то $P_1 - P_2$ — многочлен степени $\leq n - 1$ с n различными корнями, значит, он тождественно нулевой. Существование следует из единственности (линейное отображение коэффициентов в значения инъективно между пространствами одинаковой размерности n , значит, сюръективно) или прямо из формулы Ньютона ниже. $\square$

2.2.2 Разделённые разности

Определение по индукции:

$$f[x_i] = f_i, \quad f[x_i, \dots, x_{i+m}] = \frac{f[x_{i+1}, \dots, x_{i+m}] - f[x_i, \dots, x_{i+m-1}]}{x_{i+m} - x_i}.$$

Свойства, которые полезно знать на защите:

1. $f[x_1, \dots, x_m]$ — это старший коэффициент интерполяционного многочлена, построенного по узлам $x_1, \dots, x_m$;
2. разделённая разность симметрична относительно перестановки узлов (из п. 1: многочлен не зависит от порядка узлов);
3. если $f \in C^{m-1}$, то $f[x_1, \dots, x_m] = \frac{f^{(m-1)}(\xi)}{(m-1)!}$ для некоторой точки ξ из интервала, содержащего узлы;
4. для многочлена степени d все разделённые разности порядка $> d$ равны нулю.

2.2.3 Вывод формулы Ньютона

Будем добавлять узлы по одному. Пусть P_m — многочлен, интерполирующий f в узлах $x_1, \dots, x_m$. Тогда $P_{m+1} - P_m$ — многочлен степени $\leq m$, обращающийся в нуль в точках $x_1, \dots, x_m$, значит,

$$P_{m+1}(x) - P_m(x) = c_{m+1} \prod_{k=1}^m (x - x_k).$$

Подставляя $x = x_{m+1}$ и сравнивая старшие коэффициенты, получаем $c_{m+1} = f[x_1, \dots, x_{m+1}]$. Складывая всё от $m = 1$ до $n - 1$:

$$P(x) = \sum_{i=1}^n f[x_1, \dots, x_i] \prod_{k=1}^{i-1} (x - x_k) = a_1 + a_2(x - x_1) + a_3(x - x_1)(x - x_2) + \dots$$

Огромный плюс этой формы: при добавлении нового узла предыдущие коэффициенты не меняются, добавляется одно слагаемое.

2.2.4 Вычисление коэффициентов «на месте»

Таблица разделённых разностей считается прямо в массиве коэффициентов:

```
for (int i = 0; i < n; i++) coeffs[i] = f_values[i];
for (int j = 1; j < n; j++)
    for (int i = n - 1; i >= j; i--) {
        dx = x_nodes[i] - x_nodes[i - j];
        coeffs[i] = (coeffs[i] - coeffs[i - 1]) / dx;
    }
```

Внутренний цикл идёт *справа налево*: элемент `coeffs[i-1]` понадобится ещё раз на этой же итерации внешнего цикла, поэтому затирать его раньше времени нельзя. После j -го прохода `coeffs[i]` содержит $f[x_{i-j}, \dots, x_i]$; после всех проходов `coeffs[j] = f[x_1, \dots, x_{j+1}]` — ровно то, что нужно. Стоимость $O(n^2)$ действий, дополнительной памяти нет.

2.2.5 Вычисление значения: схема Горнера

$$P(x) = a_1 + (x - x_1)(a_2 + (x - x_2)(a_3 + \dots))$$

```
value = coeffs[n - 1];
for (int i = n - 2; i >= 0; i--)
    value = coeffs[i] + (x_eval - x_nodes[i]) * value;
```

Это $O(n)$ действий, без накопления длинных произведений — и быстрее, и устойчивее, чем «в лоб».

2.2.6 Погрешность и эффект Рунге

Если $f \in C^n[a, b]$, то для каждой точки x найдётся ξ такое, что

$$f(x) - P(x) = \frac{f^{(n)}(\xi)}{n!} \omega(x), \quad \omega(x) = \prod_{k=1}^n (x - x_k).$$

Два сомножителя ведут себя по-разному. Для «хороших» функций (e^x) производные растут медленно и всё сходится очень быстро. Но для $f(x) = \frac{1}{25x^2 + 1}$ на $[-1, 1]$ производные растут быстрее, чем $n!$, а $|\omega|$ на равномерной сетке у краёв отрезка в экспоненциальное число раз больше, чем в середине. В итоге при росте n приближение не улучшается, а разваливается у концов — это и есть *эффект Рунге*, его отлично видно в программе (синий график начинает болтаться у краёв). Именно поэтому в требованиях сказано: при $n > 50$ многочленное приближение не строить.

2.3 Метод 2. Кубические многочлены Эрмита, естественные граничные условия (задача 13)

2.3.1 Постановка

На каждом отрезке $[x_i, x_{i+1}]$ строится свой кубический многочлен P_i , и соседние куски склеиваются по значению и по первой производной. Кубический многочлен однозначно определяется четырьмя условиями:

$$P_i(x_i) = f_i, \quad P_i(x_{i+1}) = f_{i+1}, \quad P_i'(x_i) = d_i, \quad P_i'(x_{i+1}) = d_{i+1}.$$

Если все d_i известны, куски автоматически склеиваются: в общем узле обе формулы дают одно значение f_i и одну производную d_i , то есть $S \in C^1[a, b]$.

2.3.2 Вывод коэффициентов

Ищем $P_i(x) = c_0 + c_1 t + c_2 t^2 + c_3 t^3$, где $t = x - x_i$, $h = x_{i+1} - x_i$, $\Delta = \frac{f_{i+1} - f_i}{h}$. Из условий в левом конце сразу $c_0 = f_i$, $c_1 = d_i$. Остаются два уравнения:

$$\begin{cases} c_2 h^2 + c_3 h^3 = f_{i+1} - f_i - d_i h = h(\Delta - d_i), \\ 2c_2 h + 3c_3 h^2 = d_{i+1} - d_i. \end{cases}$$

Разделим первое на h^2 , второе на h :

$$c_2 + c_3 h = \frac{\Delta - d_i}{h}, \quad 2c_2 + 3c_3 h = \frac{d_{i+1} - d_i}{h}.$$

Вычитая из второго удвоенное первое, получаем $c_3 h = \frac{d_{i+1} + d_i - 2\Delta}{h}$, откуда

$$\boxed{c_2 = \frac{3\Delta - 2d_i - d_{i+1}}{h}, \quad c_3 = \frac{d_i + d_{i+1} - 2\Delta}{h^2}.$$

Ровно это и записано в `make_hermite_natural_coefficients`.

2.3.3 Откуда берутся d_i во внутренних узлах

Производные функции нам не даны — метод работает только по таблице значений. Во внутренних узлах используется формула через разделённые разности:

$$d_i = f[x_{i-1}, x_i] + f[x_i, x_{i+1}] - f[x_{i-1}, x_{i+1}], \quad i = 2, \dots, n-1.$$

Утверждение 2.2. Эта формула точна для любого многочлена степени ≤ 2 .

Доказательство. Для $f = 1$: $0 + 0 - 0 = 0 = f'$. Для $f = x$: $1 + 1 - 1 = 1 = f'$. Для $f = x^2$ разделённые разности первого порядка равны суммам узлов: $f[x_{i-1}, x_i] = x_{i-1} + x_i$ и т.д., поэтому $(x_{i-1} + x_i) + (x_i + x_{i+1}) - (x_{i-1} + x_{i+1}) = 2x_i = f'(x_i)$. Остальное — по линейности. $\square$

На равномерной сетке (а именно такая и строится в программе) формула превращается в обычную центральную разность $d_i = \frac{f_{i+1} - f_{i-1}}{2h}$.

2.3.4 Естественные граничные условия: вывод d_1 и d_n

В узлах x_1 и x_n соседа с одной стороны нет — это и есть «недостающие граничные условия». В задаче 13 они берутся из требования нулевой второй производной на концах (естественные условия):

$$P_1''(x_1) = 0, \quad P_{n-1}''(x_n) = 0.$$

Слева: $P_1''(x_1) = 2c_2 = \frac{2(3\Delta_1 - 2d_1 - d_2)}{h_1}$, приравнивая нулю,

$$\boxed{d_1 = \frac{3\Delta_1 - d_2}{2}}, \quad \Delta_1 = \frac{f_2 - f_1}{x_2 - x_1}.$$

Справа считаем вторую производную в правом конце последнего куска ($t = h$, $h = h_{n-1}$, $\Delta = \Delta_{n-1}$):

$$P''(x_n) = 2c_2 + 6c_3h = \frac{2(3\Delta - 2d_{n-1} - d_n)}{h} + \frac{6(d_{n-1} + d_n - 2\Delta)}{h} = \frac{2d_{n-1} + 4d_n - 6\Delta}{h},$$

откуда

$$\boxed{d_n = \frac{3\Delta_{n-1} - d_{n-1}}{2}}.$$

Поскольку d_2 и d_{n-1} уже вычислены локально, никакой системы решать не нужно: метод полностью локальный, построение стоит $O(n)$ действий.

Замечание 2.1. При $n = 2$ оба естественных условия накладываются на один и тот же кубический многочлен и заставляют его вырождаться в линейный: $d_1 = d_2 = \Delta$. Этот случай в программе обработан отдельно.

2.3.5 Точность

- **На многочленах.** Метод точен на многочленах степени 0 и 1. На квадратичных — уже нет, и это не дефект реализации: у настоящей параболы $f'' \neq 0$ на концах, а естественное условие принудительно обнуляет вторую производную сплайна. Ошибка при этом локализована в двух приграничных отрезках.
- **Порядок сходимости.** Во внутренних отрезках погрешность $O(h^3)$: производные d_i вычислены с точностью $O(h^2)$, а ошибка эрмита куска при возмущении производных на δ есть $O(h\delta)$. В приграничных отрезках d_1 определено лишь с точностью $O(h)$, поэтому там $O(h^2)$. Именно это и видно в таблицах: для функции Рунге максимум погрешности сидит внутри и отношение при удвоении n равно 8, а для e^x максимум сидит у границы и отношение равно 4.

2.3.6 Вычисление значения

Нужный отрезок ищется двоичным поиском (`std::upper_bound`) за $O(\log n)$, затем — схема Горнера:

```
i = upper_bound(x_nodes, x_nodes + n, x_eval) - x_nodes - 1;
if (i < 0) i = 0; if (i > n - 2) i = n - 2;
t = x_eval - x_nodes[i];
c = coeffs + 4 * i;
return c[0] + t * (c[1] + t * (c[2] + t * c[3]));
```

Обрезание индекса — это аккуратная экстраполяция крайним куском для точек вне $[x_1, x_n]$ (они появляются при уменьшении масштаба клавишей 3).

2.4 Метод 3. Параболические сплайны, граничные условия экстраполяцией (задача 46)

2.4.1 Почему точки склейки берутся между узлами

Посчитаем степени свободы. Пусть кусков n и на каждом стоит квадратный трёхчлен — это $3n$ неизвестных коэффициентов. Условий:

- интерполяция в n узлах — n условий;
- непрерывность значения во внутренних точках склейки — $(n - 1)$;
- непрерывность производной там же — $(n - 1)$.

Итого $3n - 2$ условия на $3n$ неизвестных: не хватает ровно двух — это и есть граничные условия. Если бы точки склейки совпали с узлами, то в каждой из них стояло бы сразу три условия (значение слева, значение справа и интерполяция), и система была бы переопределена. Поэтому точки склейки ставят *между* узлами:

$$\xi_1 = x_1 - \frac{x_2 - x_1}{2}, \quad \xi_i = \frac{x_{i-1} + x_i}{2} \ (i = 2, \dots, n), \quad \xi_{n+1} = x_n + \frac{x_n - x_{n-1}}{2}.$$

На каждом отрезке $[\xi_i, \xi_{i+1}]$ лежит ровно один узел x_i .

2.4.2 Параметризация через значения в точках склейки

Обозначим $v_i = S(\xi_i)$ — неизвестные значения сплайна в точках склейки ($n + 1$ штука). На отрезке $[\xi_i, \xi_{i+1}]$ квадратный трёхчлен однозначно определяется тремя условиями

$$P_i(\xi_i) = v_i, \quad P_i(x_i) = f_i, \quad P_i(\xi_{i+1}) = v_{i+1},$$

причём непрерывность значения при такой записи выполняется автоматически (соседние куски в точке ξ_{i+1} дают одно и то же число v_{i+1}). Введём обозначения

$$h_i = x_i - \xi_i, \quad z_i = \xi_{i+1} - \xi_i, \quad u_i = \xi_{i+1} - x_i = z_i - h_i,$$

$$p_i = \frac{f_i - v_i}{h_i}, \quad q_i = \frac{v_{i+1} - v_i}{z_i}.$$

Записывая $P_i(t) = c_0 + c_1 t + c_2 t^2$, $t = x - \xi_i$, из трёх условий получаем $c_0 = v_i$ и систему

$$c_1 h_i + c_2 h_i^2 = f_i - v_i, \quad c_1 z_i + c_2 z_i^2 = v_{i+1} - v_i,$$

то есть $c_1 + c_2 h_i = p_i$ и $c_1 + c_2 z_i = q_i$. Вычитая,

$$c_2 = \frac{q_i - p_i}{z_i - h_i} = \frac{q_i - p_i}{u_i}, \quad c_1 = p_i - c_2 h_i, \quad c_0 = v_i.$$

2.4.3 Уравнения непрерывности производной

Производная слева в точке ξ_{i+1} (конец куска номер i):

$$P'_i(\xi_{i+1}) = c_1 + 2c_2 z_i = p_i + c_2(2z_i - h_i) = \frac{f_i - v_i}{h_i} \left(1 - \frac{2z_i - h_i}{u_i}\right) + \frac{(v_{i+1} - v_i)(2z_i - h_i)}{z_i u_i}.$$

Заметим, что $1 - \frac{2z_i - h_i}{u_i} = \frac{u_i - 2z_i + h_i}{u_i} = \frac{-z_i}{u_i}$, поэтому

$$P'_i(\xi_{i+1}) = -\frac{z_i}{h_i u_i} (f_i - v_i) + \frac{2z_i - h_i}{z_i u_i} (v_{i+1} - v_i).$$

Производная справа (начало куска $i + 1$, $t = 0$):

$$P'_{i+1}(\xi_{i+1}) = c_1^{(i+1)} = p_{i+1} - c_2^{(i+1)} h_{i+1} = \frac{z_{i+1}}{h_{i+1} u_{i+1}} (f_{i+1} - v_{i+1}) - \frac{h_{i+1}}{z_{i+1} u_{i+1}} (v_{i+2} - v_{i+1}),$$

где использовано $1 + \frac{h_{i+1}}{u_{i+1}} = \frac{z_{i+1}}{u_{i+1}}$. Приравнявая и перенося известные величины вправо, получаем трёхдиагональное уравнение относительно v_i, v_{i+1}, v_{i+2} :

$$\begin{aligned} & \left[\frac{z_L}{h_L u_L} - \frac{2z_L - h_L}{z_L u_L} \right] v_i + \left[\frac{2z_L - h_L}{z_L u_L} + \frac{z_R}{h_R u_R} - \frac{h_R}{z_R u_R} \right] v_{i+1} + \left[\frac{h_R}{z_R u_R} \right] v_{i+2} \\ &= \frac{z_L f_i}{h_L u_L} + \frac{z_R f_{i+1}}{h_R u_R}, \end{aligned}$$

где индекс L относится к левому куску (номер i), R — к правому (номер $i + 1$). Таких уравнений $n - 1$ (по числу внутренних точек склейки).

2.4.4 Проверка на равномерной сетке

Пусть шаг узлов равен s , точки склейки — середины. Тогда $h_i = u_i = s/2$, $z_i = s$, и коэффициенты равны

$$\frac{z}{hu} - \frac{2z - h}{zu} = \frac{4}{s} - \frac{3}{s} = \frac{1}{s}, \quad \frac{2z - h}{zu} + \frac{z}{hu} - \frac{h}{zu} = \frac{3}{s} + \frac{4}{s} - \frac{1}{s} = \frac{6}{s}, \quad \frac{h}{zu} = \frac{1}{s},$$

правая часть $\frac{4(f_i + f_{i+1})}{s}$. После умножения на s :

$$\boxed{v_i + 6v_{i+1} + v_{i+2} = 4(f_i + f_{i+1})}$$

— классическое соотношение для квадратичного сплайна. Полезно проверить его вручную (это же и хороший ответ на защите):

- $f \equiv c$, $v \equiv c$: $c + 6c + c = 8c = 4 \cdot 2c$ — верно;
- $f = x$, $\xi_j = (j - \frac{1}{2})s$, $v_j = \xi_j$: слева $s[(i - \frac{1}{2}) + 6(i + \frac{1}{2}) + (i + \frac{3}{2})] = s(8i + 4)$, справа $4(is + (i + 1)s) = s(8i + 4)$ — верно;
- $f = x^2$, $v_j = \xi_j^2$: слева $s^2[(i - \frac{1}{2})^2 + 6(i + \frac{1}{2})^2 + (i + \frac{3}{2})^2] = s^2(8i^2 + 8i + 4)$, справа $4s^2(i^2 + (i + 1)^2) = s^2(8i^2 + 8i + 4)$ — верно.

Последняя проверка показывает, что метод воспроизводит параболы точно (при правильных граничных условиях).

2.4.5 Граничные условия: экстраполяция

Не хватает двух уравнений. В задаче 46 они берутся экстраполяцией в приграничных узлах: крайние точки склейки ξ_1 и ξ_{n+1} лежат *вне* отрезка $[x_1, x_n]$, и значение сплайна в них задаётся значением интерполяционного многочлена, построенного по трём ближайшим узлам:

$$v_1 = P_{123}(\xi_1), \quad v_{n+1} = P_{n-2, n-1, n}(\xi_{n+1}),$$

где, например,

$$P_{123}(t) = f_1 + f[x_1, x_2](t - x_1) + f[x_1, x_2, x_3](t - x_1)(t - x_2).$$

Это даёт две тривиальные строки системы ($v_1 = \text{const}$, $v_{n+1} = \text{const}$). Важное следствие: квадратичная экстраполяция точна на многочленах степени ≤ 2 , поэтому граничные условия *не портят* точность метода на параболах — в отличие от естественных условий из задачи 13. При $n = 2$ трёх узлов нет и экстраполяция делается линейно.

2.4.6 Метод прогонки

Система имеет вид $a_i v_{i-1} + b_i v_i + c_i v_{i+1} = d_i$. Ищем решение в виде $v_i = \alpha_i v_{i+1} + \beta_i$. Подставляя это представление для v_{i-1} в i -е уравнение,

$$a_i(\alpha_{i-1} v_i + \beta_{i-1}) + b_i v_i + c_i v_{i+1} = d_i \implies v_i = \frac{-c_i}{b_i + a_i \alpha_{i-1}} v_{i+1} + \frac{d_i - a_i \beta_{i-1}}{b_i + a_i \alpha_{i-1}},$$

откуда рекуррентные формулы прямого хода

$$\alpha_i = \frac{-c_i}{b_i + a_i \alpha_{i-1}}, \quad \beta_i = \frac{d_i - a_i \beta_{i-1}}{b_i + a_i \alpha_{i-1}}, \quad \alpha_0 = \frac{-c_0}{b_0}, \quad \beta_0 = \frac{d_0}{b_0},$$

и обратный ход $v_N = \beta_N$, $v_i = \alpha_i v_{i+1} + \beta_i$. Стоимость $O(n)$.

Почему прогонка здесь устойчива. На равномерной сетке строки имеют вид $(1, 6, 1)$, то есть $|b_i| = 6 > |a_i| + |c_i| = 2$ — строгое диагональное преобладание. По индукции $|\alpha_i| < 1$: при $|\alpha_{i-1}| \leq 1$

$$|\alpha_i| = \frac{|c_i|}{|b_i + a_i \alpha_{i-1}|} \leq \frac{|c_i|}{|b_i| - |a_i|} < \frac{|c_i|}{|c_i|} = 1.$$

Значит, знаменатели не обращаются в нуль и ошибки округления не нарастают; выбор главного элемента не нужен.

Экономия памяти. В реализации матрица системы нигде не хранится: коэффициенты строки вычисляются прямо в прямом ходе, массив α временно кладётся в выходной массив коэффициентов (в нём $3n$ чисел, а нужно $n + 1$), β — в рабочий массив v . Это важно для требования «должно работать при $n = 10^7$ ».

2.4.7 Вычисление значения

Отрезок $[\xi_i, \xi_{i+1}]$ симметричен относительно узла x_i , поэтому нужный кусок — тот, чей узел ближе всего к точке; он находится двоичным поиском по массиву узлов. Массив ξ при этом не нужен: левый конец куска восстанавливается по двум соседним узлам. Благодаря этому подпрограмма вычисления значения имеет ровно тот набор аргументов, который требует задание (точка, отрезок, n , узлы, коэффициенты).

2.5 Графическая часть (Qt)

2.5.1 Из чего состоит программа

Файл	Содержимое
functions.cpp/.h	семь приближаемых функций и их описания
newton_interpolation.cpp/.h	метод 1
hermite_natural.cpp/.h	метод 2
parabolic_extrapolation.cpp/.h	метод 3
window.cpp/.h	виджет: сетка, вызовы методов, масштаб, рисование
main.cpp	аргументы, окно, меню, строка состояния
graph.pro	проект для qmake

Каждая подпрограмма построения получает всё через аргументы: n , массив узлов, массив значений, массив для коэффициентов и, если нужно, рабочие массивы. Это прямое требование задания (никаких глобальных переменных и «скрытых» данных).

2.5.2 Преобразование координат

```
x_gl = (x_loc - show_a) / (show_b - show_a) * width();
y_gl = (max_y - y_loc) / (max_y - min_y) * height();
```

Отрезок $[show_a, show_b]$ растягивается ровно на ширину окна, отрезок $[min_y, max_y]$ — на высоту; масштабы по осям независимы, как и требуется. Вертикальная координата переворачивается, потому что у экрана ось y смотрит вниз.

2.5.3 Подсчёт масштаба

Функция `calculate_min_max()` пробегает столько точек, сколько пикселей по ширине, и для каждой берёт значения *только тех* графиков, которые сейчас показываются. Отсюда min_y , max_y и $\max\{|F|\} = \max(|min_y|, |max_y|)$; это число печатается и в окно, и в консоль. К границам добавляется 5% запаса, чтобы линии не прилипали к краю. При изменении размера окна вызывается `resizeEvent`, и масштаб пересчитывается заново.

2.5.4 Рисование графика

```
template <typename Function>
void Window::draw_graph(QPainter &painter, double left, double right, Function func) {
    int points = (width() > 0 ? width() : 1000);
    double delta = (right - left) / points;
    double x1 = left, y1 = func(left), x2, y2;
    for (int i = 1; i <= points; i++) {
        x2 = (i < points) ? left + i * delta : right;
        y2 = func(x2);
        if (!std::isnan(y1) && !std::isnan(y2) && !std::isinf(y1) && !std::isinf(y2))
            painter.drawLine(l2g(x1, y1), l2g(x2, y2));
        x1 = x2; y1 = y2;
    }
}
```

График считается по одной точке на пиксель — больше не нужно, меньше нельзя. Шаблонный параметр `Function` позволяет передавать сюда и указатель на функцию, и лямбду, замыкающую нужные массивы; при этом вызов инлайнится и обходится без накладных расходов.

2.5.5 Память и время при больших n

При $n = 10^7$: узлы и значения — по 80 МБ, коэффициенты Эрмита $4(n - 1)$ чисел — 320 МБ, коэффициенты параболического сплайна $3n$ — 240 МБ. Рабочие массивы выделяются локально в момент построения и сразу освобождаются. Построение — $O(n)$, вычисление значения — $O(\log n)$, поэтому перерисовка не зависит от n практически никак.

2.5.6 Что происходит при нажатии клавиши

Вся перестройка собрана в одну цепочку `rebuild()`:

```
void Window::rebuild() {
    allocate_arrays();           /* массивы под текущую n */
    calculate_points();          /* сетка + возмущение */
    build_approximations();      /* три метода */
    calculate_min_max();         /* масштаб + max|F| */
}
```

Клавиши 4 и 5 (число узлов) вызывают всю цепочку; клавиша 0 (смена функции) дополнительно пересчитывает $\max |f|$; клавиши 6 и 7 (возмущение) пропускают выделение памяти; клавиши 1, 2, 3 (режим и масштаб) не трогают коэффициенты вовсе и пересчитывают только масштаб. Это важно для больших n : поворот «ручек», не меняющих приближение, стоит порядка ширины окна операций, а не $O(n)$.

Девять режимов отображения (клавиша 1):

Режим	Что показано
0	только f
1	f и многочлен Ньютона
2	f и сплайн Эрмита
3	f и параболический сплайн
4	f и все три приближения
5	погрешность метода Ньютона
6	погрешность метода Эрмита
7	погрешность параболического сплайна
8	погрешности всех трёх методов

2.5.7 Как это выглядит

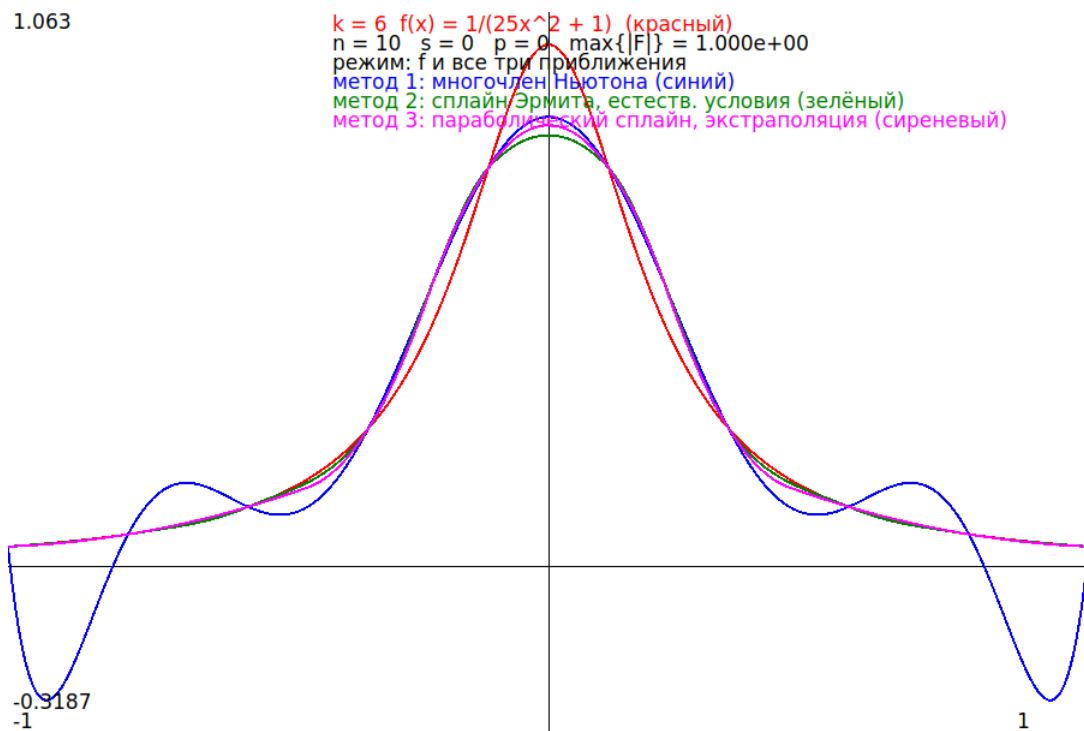


Рис. 2.1: Режим 4: функция Рунге $1/(25x^2 + 1)$ и все три приближения при $n = 10$. Красный — f , синий — многочлен Ньютона (хорошо видны колебания у краёв — эффект Рунге), зелёный — сплайн Эрмита, сиреневый — параболический сплайн.

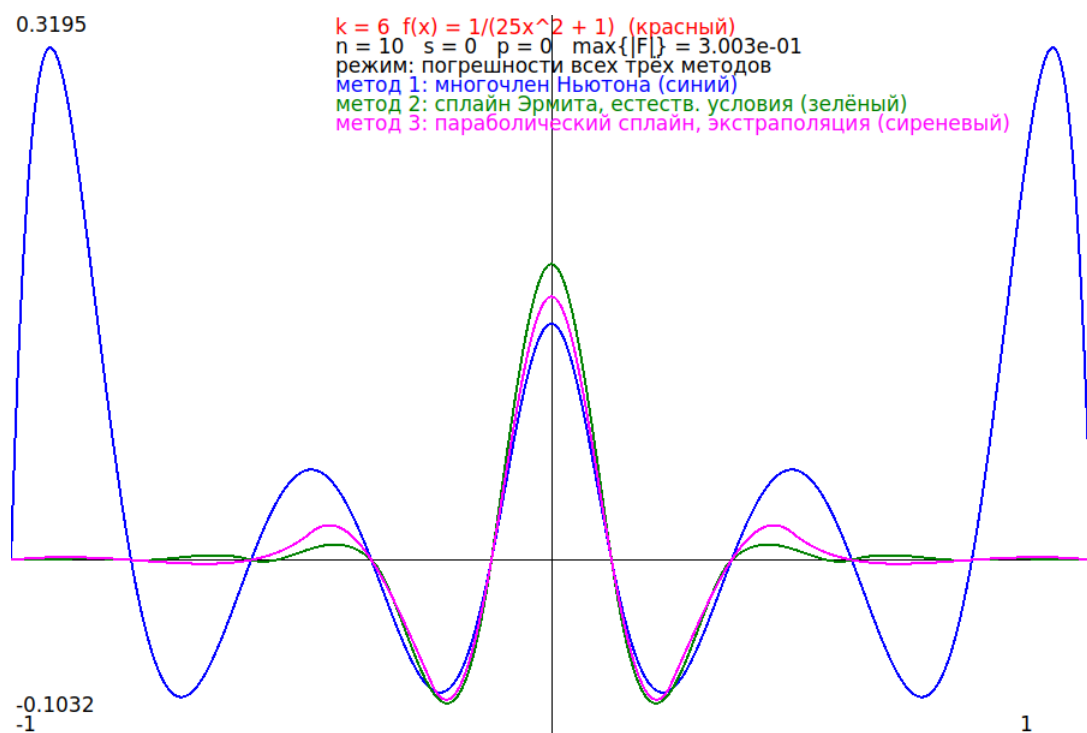


Рис. 2.2: Режим 8: погрешности всех трёх методов для той же функции. У многочлена Ньютона погрешность распределена по всему отрезку и растёт к краям, у кусочных методов сосредоточена там, где функция плохо приближается — у пика.

2.6 Проверки: что реально получилось

Точность на многочленах $([-1, 1], n = 5, \text{ максимум модуля погрешности по } 5001 \text{ точке})$:

f	Ньютон	Эрмит (13)	Параболический (46)
1	0	0	$1,1 \cdot 10^{-16}$
x	0	0	$1,1 \cdot 10^{-16}$
x^2	$1,6 \cdot 10^{-16}$	$1,9 \cdot 10^{-2}$	$1,1 \cdot 10^{-16}$
x^3	$2,4 \cdot 10^{-16}$	$7,5 \cdot 10^{-2}$	$4,3 \cdot 10^{-2}$
x^4	$6,7 \cdot 10^{-16}$	$1,4 \cdot 10^{-1}$	$1,0 \cdot 10^{-1}$

Многочлен Ньютона по пяти узлам точен на многочленах до четвёртой степени; параболический сплайн с экстраполяцией — до второй; сплайн Эрмита с естественными условиями — до первой.

Порядок сходимости , $f = \frac{1}{25x^2 + 1}, [-1, 1]$:

n	Эрмит	отношение	Параболический	отношение
50	$8,83 \cdot 10^{-4}$	—	$4,02 \cdot 10^{-4}$	—
100	$8,19 \cdot 10^{-5}$	10,8	$4,05 \cdot 10^{-5}$	9,9
200	$9,57 \cdot 10^{-6}$	8,6	$4,81 \cdot 10^{-6}$	8,4
400	$1,18 \cdot 10^{-6}$	8,1	$5,91 \cdot 10^{-7}$	8,1
800	$1,47 \cdot 10^{-7}$	8,1	$7,34 \cdot 10^{-8}$	8,1
1600	$1,83 \cdot 10^{-8}$	8,0	$9,15 \cdot 10^{-9}$	8,0
3200	$2,29 \cdot 10^{-9}$	8,0	$1,14 \cdot 10^{-9}$	8,0

Отношение 8 при удвоении n означает $O(h^3)$.

Тот же опыт для $f = e^x$ на $[0, 1]$:

n	Эрмит	отношение	Параболический	отношение
50	$4,25 \cdot 10^{-5}$	—	$1,31 \cdot 10^{-6}$	—
100	$1,03 \cdot 10^{-5}$	4,1	$1,60 \cdot 10^{-7}$	8,2
200	$2,55 \cdot 10^{-6}$	4,1	$1,98 \cdot 10^{-8}$	8,1
400	$6,33 \cdot 10^{-7}$	4,0	$2,46 \cdot 10^{-9}$	8,1
800	$1,58 \cdot 10^{-7}$	4,0	$2,99 \cdot 10^{-10}$	8,2
1600	$3,93 \cdot 10^{-8}$	4,0	$3,74 \cdot 10^{-11}$	8,0

У метода Эрмита максимум погрешности здесь достигается в приграничном отрезке, где порядок $O(h^2)$ — отсюда отношение 4. У параболического сплайна экстраполяционные граничные условия порядок не портят.

Скорость при $n = 10^7$ $([-1, 1], f = 1/(25x^2 + 1))$: построение сетки 0,16 с, построение сплайна Эрмита 0,45 с, параболического 0,63 с, вычисление 1200 точек двух графиков 0,0015 с. Максимальная погрешность обоих методов при таком n равна $2 \cdot 10^{-16}$ и $4 \cdot 10^{-16}$ — машинная точность.

2.7 Вопросы, которые обычно задают

Почему многочлен Ньютона, а не Лагранжа? Это один и тот же многочлен, разные формы записи. Форма Ньютона удобнее: добавление узла не требует пересчёта, коэффициенты считаются за $O(n^2)$ и хранятся, а значение — за $O(n)$ по схеме Горнера.

Почему при $n > 50$ многочлен не строится? Из-за эффекта Рунге на равномерной сетке приближение при больших n не улучшается, а разваливается; плюс стоимость $O(n^2)$. Это прямое требование задания.

Точен ли метод 13 на параболе? Нет, и это принципиально: естественные условия требуют $S'' = 0$ на концах, а у параболы вторая производная постоянна и отлична от нуля. Точность — только на многочленах степени ≤ 1 , зато метод полностью локальный.

Почему у метода 13 отношение погрешностей то 8, то 4? Внутри отрезка порядок $O(h^3)$, в двух приграничных отрезках — $O(h^2)$. Какое из двух чисел окажется максимумом, зависит от функции, поэтому на $1/(25x^2 + 1)$ видно 8, а на $e^x - 4$.

Зачем в задаче 46 точки склейки между узлами? Иначе система переопределена: $3n$ неизвестных и $3n$ условий плюс два граничных. Смещение склеек на полшага даёт ровно две недостающие степени свободы.

Почему прогонка без выбора главного элемента? Матрица системы строго диагонально доминирующая (1, 6, 1 на равномерной сетке), поэтому все $|\alpha_i| < 1$ и схема устойчива.

Что происходит при нажатии клавиши 6 (возмущение)? Значение в средней точке заменяется на $f(x_{n/2}) + p \cdot 0,1 \max |f|$, и все приближения строятся заново по испорченным данным. Хорошо видно принципиальное различие: многочлен Ньютона размазывает одну ошибку по всему отрезку, а кусочные методы локализуют её в нескольких соседних отрезках.

Почему при $n = 10^7$ всё быстро? Построение обоих кусочных методов линейно по n , а рисование требует лишь по одной точке на пиксель, и каждая стоит $O(\log n)$ (двоичный поиск отрезка).

Глава 3

Задание 3. Интерполяция функции двух переменных по формуле Ньютона

3.1 Постановка

Дана функция $f(x, y)$ на прямоугольнике $[a, b] \times [c, d]$ и сетка узлов $x_1, \dots, x_{n_x}$ по первой переменной, $y_1, \dots, y_{n_y}$ — по второй. Нужно построить интерполяционный многочлен по формуле Ньютона (в двумерном случае это интерполяция *тензорными произведениями*) и показать в окне три поверхности: саму функцию, приближение и погрешность.

Набор приближаемых функций:

$$\begin{aligned} k = 0 : 1, & \quad k = 1 : x, & \quad k = 2 : y, & \quad k = 3 : x + y, \\ k = 4 : \sqrt{x^2 + y^2}, & \quad k = 5 : x^2 + y^2, & \quad k = 6 : e^{x^2 - y^2}, & \quad k = 7 : \frac{1}{25(x^2 + y^2) + 1}. \end{aligned}$$

Запуск: `./graph3d a b c d nx ny mx my k`, где n_x, n_y — узлы интерполяции, m_x, m_y — сетка триангуляции, по которой рисуется поверхность (на само приближение она не влияет).

3.2 Математика: тензорные произведения

3.2.1 Базис

Как в конспекте (§1.5.3), берём одномерные базисы Ньютона по каждой переменной:

$$u_i(x) = \prod_{k=1}^{i-1} (x - x_k), \quad i = 1, \dots, n_x, \quad v_j(y) = \prod_{l=1}^{j-1} (y - y_l), \quad j = 1, \dots, n_y$$

(пустое произведение равно единице, так что $u_1 \equiv v_1 \equiv 1$). Базис в пространстве функций двух переменных — тензорные произведения

$$w_{ij}(x, y) = u_i(x) v_j(y) = \prod_{k=1}^{i-1} \prod_{l=1}^{j-1} (x - x_k)(y - y_l),$$

а приближающая функция ищется в виде

$$P(x, y) = \sum_{i=1}^{n_x} \sum_{j=1}^{n_y} c_{ij} u_i(x) v_j(y).$$

Функционалы — значения в узлах; матрицы A и B из общей схемы единичные, поэтому коэффициенты равны смешанным (двойным) разделённым разностям

$$c_{ij} = f[x_1, \dots, x_i; y_1, \dots, y_j].$$

Смешанная разделённая разность — это обычная разделённая разность по x , применённая к разделённой разности по y (и наоборот: порядок не важен, так как обе операции линейны и действуют по разным индексам).

3.2.2 Почему это действительно интерполяция

Утверждение 3.1. $P(x_l, y_m) = f(x_l, y_m)$ для всех $l = 1, \dots, n_x$, $m = 1, \dots, n_y$.

Доказательство. Перепишем сумму как $P(x, y) = \sum_i u_i(x) g_i(y)$, где $g_i(y) = \sum_j f[x_1, \dots, x_i; y_1, \dots, y_j] v_j(y)$. При фиксированном i выражение g_i — это одномерный интерполяционный многочлен Ньютона (по переменной y) для функции $y \mapsto f[x_1, \dots, x_i; y]$, то есть для разделённой разности по x , рассматриваемой как функция y . По одномерной теореме об интерполяции $g_i(y_m) = f[x_1, \dots, x_i; y_m]$ для всех m .

Теперь при фиксированном m функция $x \mapsto P(x, y_m) = \sum_i u_i(x) f[x_1, \dots, x_i; y_m]$ — это одномерный многочлен Ньютона для функции $x \mapsto f(x, y_m)$, а значит, $P(x_l, y_m) = f(x_l, y_m)$. $\square$

Утверждение 3.2. Если f — многочлен степени $\leq n_x - 1$ по x и $\leq n_y - 1$ по y , то $P \equiv f$.

Доказательство. Оба одномерных шага в доказательстве выше точны на многочленах соответствующих степеней. $\square$

3.2.3 Алгоритм: два прохода по таблице

Коэффициенты считаются прямо в массиве, в два прохода.

Шаг 0. $c_{ij} = f(x_i, y_j)$.

Шаг 1 (по x). Для $\text{level} = 1, \dots, n_x - 1$ и для $i = n_x - 1$ вниз до level :

$$c_{ij} \leftarrow \frac{c_{ij} - c_{i-1,j}}{x_i - x_{i-\text{level}}} \quad \text{сразу для всех } j.$$

После этого в строке i стоят разделённые разности по x порядка $i - 1$.

Шаг 2 (по y). То же самое внутри каждой строки: для $\text{level} = 1, \dots, n_y - 1$ и $j = n_y - 1$ вниз до level :

$$c_{ij} \leftarrow \frac{c_{ij} - c_{i,j-1}}{y_j - y_{j-\text{level}}}.$$

Движение «сверху вниз» по индексу (от большого к меньшему) нужно ровно по той же причине, что и в одномерном случае: значение $c_{i-1,j}$ ещё понадобится. Сложность: $O(n_x^2 n_y + n_x n_y^2)$ действий и $O(n_x n_y)$ памяти (дополнительной памяти нет вовсе, всё делается на месте).

Соответствующий код:

```
for (int level = 1; level < nx; level++)
    for (int i = nx - 1; i >= level; i--) {
        dx = x_nodes[i] - x_nodes[i - level];
        for (int j = 0; j < ny; j++)
            coeffs[i*ny + j] = (coeffs[i*ny + j] - coeffs[(i-1)*ny + j]) / dx;
    }
for (int i = 0; i < nx; i++)
    for (int level = 1; level < ny; level++)
        for (int j = ny - 1; j >= level; j--) {
            dy = y_nodes[j] - y_nodes[j - level];
            coeffs[i*ny + j] = (coeffs[i*ny + j] - coeffs[i*ny + (j-1)]) / dy;
        }
```

Обратите внимание на порядок циклов в первом проходе: внутренний цикл по j идёт по подряд лежащей памяти (хранение построчное, $f[i*ny+j]$), поэтому кэш используется эффективно.

3.2.4 Вычисление значения: двойная схема Горнера

$$P(x, y) = g_1(y) + (x - x_1)(g_2(y) + (x - x_2)(\dots)), \quad g_i(y) = c_{i1} + (y - y_1)(c_{i2} + (y - y_2)(\dots)).$$

```
result = coeffs[(nx-1)*ny + ny-1];
for (int j = ny - 2; j >= 0; j--)
    result = result * (py - y_nodes[j]) + coeffs[(nx-1)*ny + j];
for (int i = nx - 2; i >= 0; i--) {
    inner = coeffs[i*ny + ny-1];
    for (int j = ny - 2; j >= 0; j--)
        inner = inner * (py - y_nodes[j]) + coeffs[i*ny + j];
    result = result * (px - x_nodes[i]) + inner;
}
```

Стоимость $O(n_x n_y)$ на одну точку; произведения $u_i(x)$, $v_j(y)$ явно не вычисляются.

3.3 Выбор узлов: Чебышёв и порядок Лежа

3.3.1 Почему не равномерные узлы

В двумерном случае эффект Рунге никуда не девается (он есть уже по каждой переменной). Поэтому узлы берутся чебышёвские — нули многочлена Чебышёва:

$$t_i = \frac{a+b}{2} + \frac{b-a}{2} \cos \frac{\pi(i + \frac{1}{2})}{n}, \quad i = 0, \dots, n-1.$$

Для них величина $\max |\omega(x)|$ минимальна среди всех расположений узлов, и интерполяция ведёт себя хорошо. Проверка (функция $1/(25(x^2 + y^2) + 1)$, $n_x = n_y = 30$, максимум погрешности по сетке 150×150):

равномерные узлы	$1,77 \cdot 10^6$
узлы Чебышёва	$9,93 \cdot 10^{-3}$

3.3.2 Почему важен порядок узлов (тонкое место)

Сам многочлен от порядка узлов не зависит — он единственный. Но *схема вычислений* зависит: разделённые разности высокого порядка при монотонном (по возрастанию) порядке узлов содержат огромные взаимно сокращающиеся слагаемые, и ошибки округления растут лавинообразно. Лечение — перестановка узлов в последовательность Лежа: очередным берётся узел, максимизирующий произведение расстояний до уже выбранных,

$$t_k = \arg \max_t \prod_{m < k} |t - t_m|.$$

Реализуется за $O(n^2)$ с инкрементальным пересчётом весов:

```
for (int i = 0; i < n; i++) weight[i] = fabs(nodes[i] - center);
for (int k = 0; k < n - 1; k++) {
    /* pick the largest weight among the rest and swap it into position k */
    for (int i = k + 1; i < n; i++)
        weight[i] *= fabs(nodes[i] - nodes[k]);
}
```

Что это даёт (максимум $|f - P|$, узлы Чебышёва, $[-1, 1]^2$):

n	$f = e^{x^2-y^2}$		$f = 1/(25(x^2 + y^2) + 1)$	
	монотонный	Лежа	монотонный	Лежа
16	$2,66 \cdot 10^{-9}$	$2,66 \cdot 10^{-9}$	$1,42 \cdot 10^{-1}$	$1,42 \cdot 10^{-1}$
24	$2,33 \cdot 10^{-13}$	$2,22 \cdot 10^{-15}$	—	—
32	$2,49 \cdot 10^{-5}$	$2,22 \cdot 10^{-15}$	$7,97 \cdot 10^1$	$6,72 \cdot 10^{-3}$
40	$1,33 \cdot 10^3$	$2,22 \cdot 10^{-15}$	$1,86 \cdot 10^6$	$1,39 \cdot 10^{-3}$
50	$2,47 \cdot 10^{13}$	$3,77 \cdot 10^{-15}$	$8,11 \cdot 10^{11}$	$1,93 \cdot 10^{-4}$
64	$5,23 \cdot 10^{27}$	$5,55 \cdot 10^{-15}$	$1,73 \cdot 10^{25}$	$1,20 \cdot 10^{-5}$

Без перестановки программа при $n_x = n_y = 32$ уже рисует мусор; с перестановкой всё остаётся на уровне машинной точности. Это хороший ответ на вопрос «а почему у вас узлы лежат в странном порядке».

3.4 Трёхмерная графика без OpenGL

3.4.1 Что рисуется

Поверхность $z = F(x, y)$, где F — либо f , либо P , либо $f - P$ (переключается клавишей 1). Область разбивается сеткой $m_x \times m_y$ точек; в каждой считается значение, попутно находятся $\min z$ и $\max z$ и выводится $\max\{|F|\}$.

3.4.2 Нормировка и проекция

Координаты приводятся к кубу $[-1, 1]^3$:

$$\tilde{x} = \frac{x - c_x}{h_x}, \quad \tilde{y} = \frac{y - c_y}{h_y}, \quad \tilde{z} = 0,8 \left(2 \frac{z - z_{\min}}{z_{\max} - z_{\min}} - 1 \right),$$

где c_x, h_x — центр и полуширина отображаемого прямоугольника. Благодаря этому картинка никогда не выходит за окно и никогда не оказывается слишком мелкой, а масштабы по осям независимы.

Камера задаётся двумя углами: φ — поворот вокруг вертикальной оси (клавиши 8 и 9), θ — подъём над плоскостью (клавиши Up/Down). Оорты системы координат наблюдателя:

$$\mathbf{r} = (-\sin \varphi, \cos \varphi, 0), \quad \mathbf{u} = (-\sin \theta \cos \varphi, -\sin \theta \sin \varphi, \cos \theta), \quad \mathbf{n} = (\cos \theta \cos \varphi, \cos \theta \sin \varphi, \sin \theta).$$

Легко проверить, что тройка $(\mathbf{r}, \mathbf{u}, \mathbf{n})$ ортонормирована. Экранные координаты точки $\mathbf{p}$ — это скалярные произведения $(\mathbf{p}, \mathbf{r})$ и $(\mathbf{p}, \mathbf{u})$, а глубина — $(\mathbf{p}, \mathbf{n})$: чем больше, тем ближе к наблюдателю. Проекция ортогональная (параллельная), поэтому прямоугольная сетка остаётся сеткой и картинка читается лучше, чем при перспективе.

```

sx = -nX*sin(phi) + nY*cos(phi);
sy = -(nX*cos(phi) + nY*sin(phi))*sin(theta) + nZ*cos(theta);
depth = (nX*cos(phi) + nY*sin(phi))*cos(theta) + nZ*sin(theta);
scale = 0.32*min(width(), height());
return QPointF(0.5*width() + scale*sx, 0.5*height() - scale*sy);

```

3.4.3 Видимость: алгоритм художника

Задача «что кого закрывает» решается самым простым надёжным способом: все клетки сетки сортируются по глубине своего центра и рисуются от дальних к ближним, ближние закрывают дальние. Для графика функции (каждой точке (x, y) отвечает ровно одно z) этого достаточно — взаимных пересечений у клеток нет. Стоимость кадра $O(m_x m_y \log(m_x m_y))$.

Цвет клетки зависит от её средней высоты: синий $\rightarrow$ голубой $\rightarrow$ зелёный $\rightarrow$ жёлтый $\rightarrow$ красный. При небольших сетках дополнительно рисуются рёбра, при больших они отключаются, иначе картинка превращается в сплошную сетку.

3.5 Разбор программы

Файл	Содержимое
functions2d.cpp/.h	восемь функций $f(x, y)$ и их описания
newton2d.cpp/.h	узлы, порядок Лежа, коэффициенты, значение, погрешность
window3d.cpp/.h	сетка, проекция, рисование, клавиши
main.cpp	аргументы, окно, меню, строка состояния
graph3d.pro	проект для qmake

Ключевые функции метода:

```
void make_chebyshev_nodes(int n, double a, double b, double *nodes);
void reorder_nodes_leja(int n, double *nodes);
int make_newton2d_coefficients(int nx, int ny, const double *x, const double *y,
                               const double *f, double *coeffs);
double calculate_newton2d_value(double px, double py, int nx, int ny,
                                const double *x, const double *y, const double *coeffs);
double calculate_newton2d_discrepancy(double px, double py, func2_t func, int nx, int ny,
                                       const double *x, const double *y, const double *coeffs);
```

Значения и коэффициенты хранятся построчно: $f[i*ny+j]$ отвечает узлу (x_i, y_j) . Возмущение (клавиши 6 и 7) добавляется к значению в центральном узле, после чего коэффициенты строятся заново.

Виджет Window3D делает три вещи: `build_interpolation()` (узлы $\rightarrow$ порядок Лежа $\rightarrow$ значения $\rightarrow$ коэффициенты), `build_mesh()` (значения на сетке рисования, $\min z$, $\max z$, $\max |F|$) и `paintEvent()` (каркас, поверхность, надписи). Каждое нажатие клавиши вызывает ровно те этапы, которые действительно изменились: смена режима не пересчитывает коэффициенты, поворот камеры не пересчитывает даже сетку.

3.6 Как это выглядит

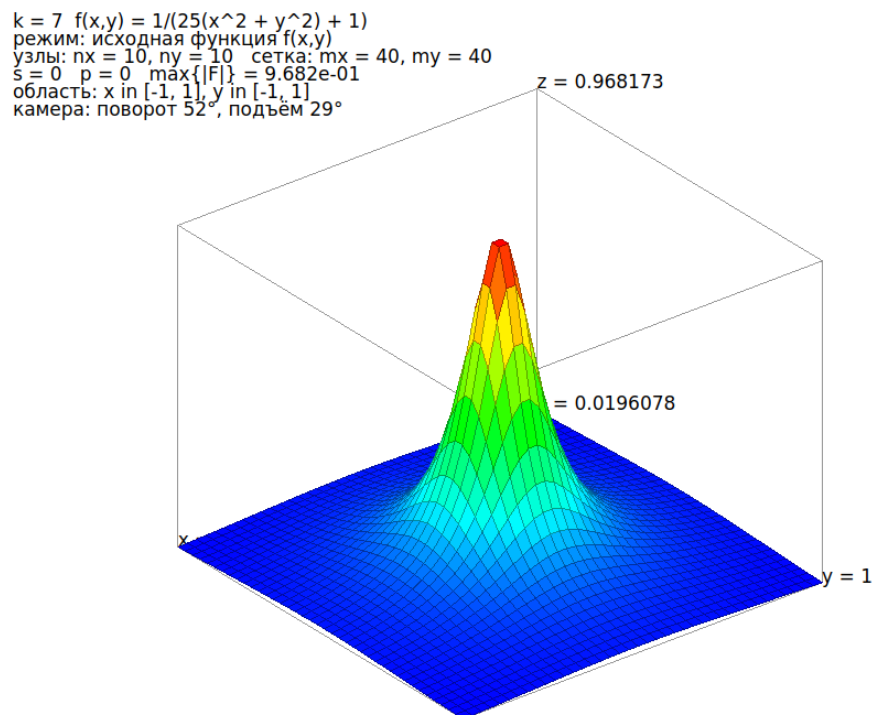


Рис. 3.1: Режим «функция»: $f(x, y) = 1/(25(x^2 + y^2) + 1)$ на $[-1, 1]^2$. В левом верхнем углу — вся служебная информация: номер функции, режим, n_x, n_y, m_x, m_y , масштаб, возмущение, $\max |F|$ и углы камеры.

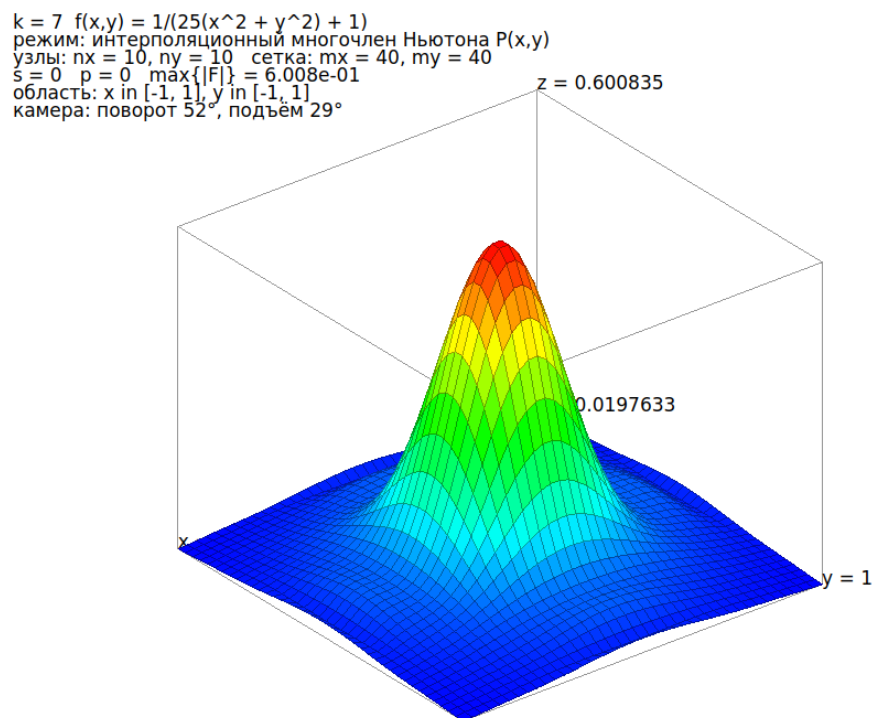


Рис. 3.2: Режим «приближение»: интерполяционный многочлен Ньютона по сетке 10×10 узлов. Пик занижен — десяти узлов на такую функцию не хватает.

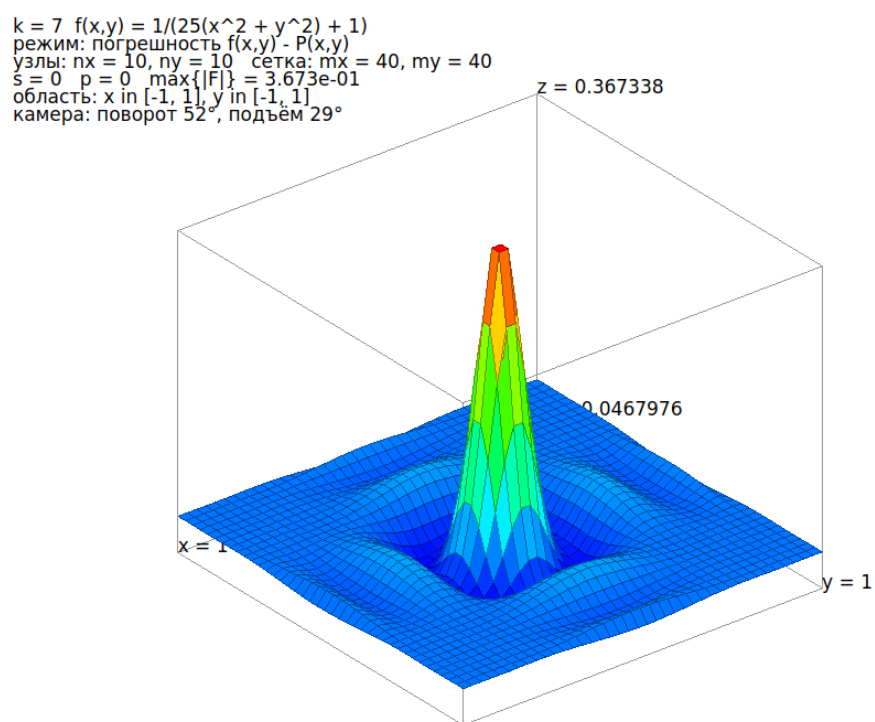


Рис. 3.3: Режим «погрешность» $f - P$: максимум ошибки сидит ровно на пике, по краям области поверхность почти плоская.

3.7 Проверки: что реально получилось

1. Интерполяция точна в узлах. Для всех восьми функций при $n_x = n_y = 5$ максимум $|P(x_i, y_j) - f(x_i, y_j)|$ по всем узлам не превосходит 10^{-15} .

2. Точность на многочленах ($[-1, 1]^2$, $n_x = n_y = 5$, максимум по сетке 120×120):

f	$\max f - P $
1	0
x	$1,1 \cdot 10^{-16}$
y	$1,1 \cdot 10^{-16}$
$x + y$	$4,4 \cdot 10^{-16}$
$x^2 + y^2$	$8,9 \cdot 10^{-16}$
$\sqrt{x^2 + y^2}$	$1,3 \cdot 10^{-1}$ (негладкая в нуле)
$e^{x^2 - y^2}$	$2,4 \cdot 10^{-2}$
$1/(25(x^2 + y^2) + 1)$	$4,1 \cdot 10^{-1}$

3. Сходимость ($[-1, 1]^2$, узлы Чебышёва в порядке Лежа):

$n = n_x = n_y$	$e^{x^2 - y^2}$	$1/(25(x^2 + y^2) + 1)$
4	$2,42 \cdot 10^{-1}$	$8,43 \cdot 10^{-1}$
8	$1,20 \cdot 10^{-3}$	$5,31 \cdot 10^{-1}$
16	$2,66 \cdot 10^{-9}$	$1,42 \cdot 10^{-1}$
32	$2,66 \cdot 10^{-15}$	$6,72 \cdot 10^{-3}$
50	$2,89 \cdot 10^{-15}$	$1,93 \cdot 10^{-4}$

Для аналитической функции сходимость экспоненциальная — машинная точность уже при 32 узлах. Для функции с «пиком» сходимость медленнее, но она есть (на равномерных узлах её бы не было вовсе).

4. Скорость. Самый тяжёлый режим ($n_x = n_y = 64$, сетка 128×128): построение коэффициентов — меньше 1 мс, полный пересчёт поверхности (16 384 точки по $O(n_x n_y)$ каждая) — 0,22 с, отрисовка кадра — 0,04 с.

5. Устойчивость интерфейса. Прогон, в котором для всех восьми функций последовательно нажимаются все клавиши (режимы, узлы, сетка, масштаб, возмущение, повороты) — около 400 перестроений — проходит без падений, без NaN и без зависаний.

3.8 Вопросы, которые обычно задают

Почему это «формула Ньютона», а не просто двумерная интерполяция? Потому что базис и коэффициенты — ньютоновские по каждой переменной: $w_{ij} = u_i v_j$, $c_{ij} = f[x_1..x_i; y_1..y_j]$. Это буквально тензорное произведение двух одномерных формул Ньютона.

Важен ли порядок проходов (сначала по x , потом по y)? Нет. Обе операции линейны и действуют по разным индексам, поэтому коммутуют; результат одинаков.

Какова степень получающегося многочлена? Не выше $n_x - 1$ по x и $n_y - 1$ по y ; полная степень может достигать $n_x + n_y - 2$. Он точен на всех таких многочленах.

Почему нельзя взять равномерные узлы? Эффект Рунге: при $n_x = n_y = 30$ погрешность на равномерных узлах $\sim 10^6$ против 10^{-2} на чебышёвских.

Зачем перестановка Лежа? Это чисто вычислительный приём: он не меняет многочлен, но делает вычисление разделённых разностей и схему Горнера устойчивыми. Без него при $n \geq 32$ результат разваливается (см. таблицу).

Почему алгоритм художника, а не z -буфер? Для графика функции клетки не пересекаются, поэтому сортировки по глубине достаточно; это проще и не требует памяти под буфер глубины.

Что показывает режим погрешности при возмущении? Что многочленная интерполяция *нелокальна*: испорченное значение в одном узле меняет приближение по всей области. У кусочных методов (задание 2) такого нет.

Приложение А

Сборка: какие библиотеки нужны

А.1 Задание 1 (MPI)

```
sudo apt install build-essential openmpi-bin libopenmpi-dev # Ubuntu/Debian
brew install open-mpi # macOS
```

Сборка:

```
cd task1_yarik
make # or, in a single command:
mpicc -O2 -Wall -Wextra main.c input.c output.c gauss_reverse_MPI.c -o a.out -lm
mpirun -np 4 ./a.out 1000 5 1
```

А.2 Задания 2 и 3 (Qt)

```
sudo apt install build-essential qtbase5-dev qtbase5-dev-tools qt5-qmake # Qt5
sudo apt install build-essential qt6-base-dev qt6-base-dev-tools # Qt6
brew install qt # macOS
```

Сборка и запуск:

```
cd task2_yarik && qmake graph.pro && make && ./graph -1 1 10 6
cd task3_yarik && qmake graph3d.pro && make && ./graph3d -1 1 -1 1 10 10 40 40 7
```

Если **qmake** не находится, он может называться **qmake6** или **qmake-qt5**. Сборка без **qmake** (нужно самому вызвать **moc**) описана в файлах **разбор.txt** в каждой папке.

Приложение В

Шпаргалка по клавишам

В.1 Задание 2

Клавиша	Действие
0	следующая функция f_k (возмущение сбрасывается)
1	следующий набор графиков (9 режимов)
2 / 3	увеличить / уменьшить масштаб по x вдвое
4 / 5	удвоить / уменьшить вдвое число узлов n
6 / 7	увеличить / уменьшить возмущение p
Ctrl+Q	выход

В.2 Задание 3

Клавиша	Действие
0	следующая функция f_k
1	режим: функция $\rightarrow$ приближение $\rightarrow$ погрешность
2 / 3	увеличить / уменьшить масштаб области
4 / 5	удвоить / уменьшить вдвое n_x, n_y
6 / 7	увеличить / уменьшить возмущение p
8 / 9	повернуть камеру влево / вправо
+ / -	удвоить / уменьшить вдвое сетку рисования m_x, m_y
Up / Down	поднять / опустить камеру
Ctrl+Q	выход

Приложение С

Частые ошибки и как их чинить

`mpi.h: No such file or directory` не установлен `libopenmpi-dev`, либо сборка идёт через `gcc` вместо `mpicc`.

Программа на MPI работает медленнее на большем числе процессов процессов больше, чем физических ядер. Смотрите `mpirun`; на двух ядрах запуск с `-np 4` даёт замедление в несколько раз.

`qmake: command not found` в новых Ubuntu команда называется `qmake6`; либо не установлен пакет `qt5-qmake`.

Ошибки вида `undefined reference to 'vtable for Window'` не запущен `moc` для заголовка с макросом `Q_OBJECT` (при сборке через `qmake` это делается автоматически).

График «прыгает» при изменении размера окна так и задумано: масштаб по вертикали пересчитывается по реально видимым графикам, а число точек выборки равно числу пикселей.

При больших n многочлен Ньютона не рисуется это не ошибка: при $n > 50$ многочленное приближение по заданию не строится.

Приложение D

Полные тексты файлов с методами

Ниже приведены целиком те файлы, в которых живёт сама численная математика — именно о них чаще всего спрашивают на защите. Графические файлы (`window.cpp`, `window3d.cpp`, `main.cpp`) разобраны в тексте по частям, их полные версии лежат в репозитории.

D.1 Задание 1: `gauss_reverse_MPI.c`

```
#include <stdio.h>
#include "gauss_reverse_MPI.h"

/* Меняет местами глобальные строки i и j распределённой матрицы.
   Если строки живут на разных процессах, они просто обмениваются ими. */
static void swap_rows_mpi(double *local_m, int n, int i, int j, int rank, int size)
{
    int owner_i = i%size, owner_j = j%size, t = 0;
    double tmp = 0;

    if (owner_i == owner_j) {
        if (rank == owner_i) {
            for (t = 0; t < n; ++t) {
                tmp = local_m[(i/size)*n+t];
                local_m[(i/size)*n+t] = local_m[(j/size)*n+t];
                local_m[(j/size)*n+t] = tmp;
            }
        }
    } else if (rank == owner_i) {
        MPI_Sendrecv_replace(local_m+(i/size)*n, n, MPI_DOUBLE, owner_j, 0,
                             owner_j, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    } else if (rank == owner_j) {
        MPI_Sendrecv_replace(local_m+(j/size)*n, n, MPI_DOUBLE, owner_i, 0,
                             owner_i, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    }
}

int gauss_reverse_mpi(int n, double *local_a, double *local_x, int rank, int size,
                     double *buffer, int *col)
{
    int local_n = 0, k = 0, i = 0, j = 0, gi = 0, lk = 0, pi = 0, pj = 0, t = 0;
    double eps = 0, norm = 0, c = 0, inv = 0, tmp = 0;
    double *row_a = buffer, *row_x = buffer + n;
    MAX local_max, global_max;

    local_n = n/size;
    if (rank < n%size)
        ++local_n;

    /* Норма матрицы: максимум по столбцам суммы модулей.
       Нужна только для того, чтобы понимать, какое число уже считать нулём. */
```

```

for (j = 0; j < n; ++j)
    row_a[j] = 0;
for (i = 0; i < local_n; ++i)
    for (j = 0; j < n; ++j)
        row_a[j] += fabs(local_a[i*n+j]);
MPI_Allreduce(row_a, row_x, n, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
for (j = 0; j < n; ++j)
    if (row_x[j] > norm)
        norm = row_x[j];
eps = 1e-14*norm;

/* X = E, col = (0, 1, ..., n-1) */
for (i = 0; i < local_n; ++i)
    for (j = 0; j < n; ++j)
        local_x[i*n+j] = (i*size+rank == j ? 1. : 0.);
for (i = 0; i < n; ++i)
    col[i] = i;

/* ----- ПРЯМОЙ ХОД ----- */
for (k = 0; k < n; ++k) {
    /* 1. Главный элемент - самый большой по модулю во всей оставшейся
       подматрице (строки и столбцы с номерами от k до n-1). */
    local_max.value = -1;
    local_max.index = 0;
    for (i = 0; i < local_n; ++i) {
        gi = i*size + rank;
        if (gi < k)
            continue;
        for (j = k; j < n; ++j) {
            tmp = fabs(local_a[i*n+j]);
            if (tmp > local_max.value) {
                local_max.value = tmp;
                local_max.index = gi*n + j;
            }
        }
    }
    MPI_Allreduce(&local_max, &global_max, 1, MPI_DOUBLE_INT, MPI_MAXLOC, MPI_COMM_WORLD);

    if (global_max.value <= eps)
        return -1; /* матрица вырождена */

    pi = global_max.index/n;
    pj = global_max.index - pi*n;

    /* 2. Ставим главный элемент в угол (k, k).
       Строки переставляем и в A, и в X, столбцы - только в A. */
    if (pi != k) {
        swap_rows_mpi(local_a, n, k, pi, rank, size);
        swap_rows_mpi(local_x, n, k, pi, rank, size);
    }
    if (pj != k) {
        t = col[k];
        col[k] = col[pj];
        col[pj] = t;
        for (i = 0; i < local_n; ++i) {
            tmp = local_a[i*n+k];
            local_a[i*n+k] = local_a[i*n+pj];
            local_a[i*n+pj] = tmp;
        }
    }

    /* 3. Делим строку k на главный элемент и рассылаем её всем процессам:
       в row_a - строка матрицы A, в row_x - строка матрицы X. */
    if (rank == k%size) {
        lk = k/size;

```

```

        inv = 1./local_a[lk*n+k];
        for (j = k; j < n; ++j)
            local_a[lk*n+j] *= inv;
        for (j = 0; j < n; ++j)
            local_x[lk*n+j] *= inv;
        for (j = 0; j < n; ++j) {
            row_a[j] = local_a[lk*n+j];
            row_x[j] = local_x[lk*n+j];
        }
    }
    MPI_Bcast(buffer, 2*n, MPI_DOUBLE, k%size, MPI_COMM_WORLD);

    /* 4. Вычитаем строку k только из строк НИЖЕ неё - это и есть Гаусс
       (у Жордана здесь зануляются и строки выше). */
    for (i = 0; i < local_n; ++i) {
        gi = i*size + rank;
        if (gi <= k)
            continue;
        c = local_a[i*n+k];
        if (c == 0.)
            continue;
        for (j = k; j < n; ++j)
            local_a[i*n+j] -= c*row_a[j];
        for (j = 0; j < n; ++j)
            local_x[i*n+j] -= c*row_x[j];
    }
}

/* ----- ОБРАТНЫЙ ХОД ----- */
/* Сейчас A - верхняя треугольная с единицами на диагонали.
   Идём снизу вверх и убираем то, что осталось над диагональю.
   Саму матрицу A при этом не трогаем: она дальше не нужна. */
for (k = n-1; k > 0; --k) {
    if (rank == k%size) {
        lk = k/size;
        for (j = 0; j < n; ++j)
            row_x[j] = local_x[lk*n+j];
    }
    MPI_Bcast(row_x, n, MPI_DOUBLE, k%size, MPI_COMM_WORLD);

    for (i = 0; i < local_n; ++i) {
        gi = i*size + rank;
        if (gi >= k)
            continue;
        c = local_a[i*n+k];
        if (c == 0.)
            continue;
        for (j = 0; j < n; ++j)
            local_x[i*n+j] -= c*row_x[j];
    }
}

/* ----- РАССТАНОВКА СТРОК НА МЕСТА ----- */
/* Из-за перестановок столбцов строки ответа перепутаны:
   строка k того, что получилось, - это строка col[k] обратной матрицы.
   Массив col одинаковый на всех процессах, поэтому все делают одни и те же обмены. */
for (i = 0; i < n; ++i) {
    while (col[i] != i) {
        j = col[i];
        swap_rows_mpi(local_x, n, i, j, rank, size);
        col[i] = col[j];
        col[j] = i;
    }
}
}

```



```

    return 0;
}

/* ||A*B - E||_1 = максимум по столбцам суммы модулей.
   Чтобы не заводить лишнюю память, куски столбца матрицы B гоняются по кольцу
   процессов: за size пересылок каждый процесс увидит весь столбец целиком
   и вернёт свой кусок обратно. */
void r_mpi(int n, double *local_a, double *local_b, double *r, int rank, int size, double *buffer)
{
    int i = 0, j = 0, k = 0, t = 0, local_n = 0, tmp_n = 0, new_rank = 0;
    double sum = 0, col_sum = 0, max = -1;
    MPI_Datatype col_t;

    local_n = n/size;
    if (rank < n%size)
        ++local_n;

    /* столбец локальной матрицы: n/size+1 чисел с шагом n */
    MPI_Type_vector(n/size+1, 1, n, MPI_DOUBLE, &col_t);
    MPI_Type_commit(&col_t);

    for (t = 0; t < n; ++t) {
        /* buffer[i] = (A*B)[global_row(i)][t] - E[global_row(i)][t] */
        for (i = 0; i < local_n; ++i)
            buffer[i] = (i*size+rank == t ? -1. : 0.);

        for (k = 0; k < size; ++k) {
            new_rank = (rank+k)%size;
            tmp_n = n/size;
            if (new_rank < n%size)
                ++tmp_n;

            /* сейчас у нас куски столбца t, которые принадлежат процессу new_rank;
               складываем их подряд в buffer+n, чтобы читать память по порядку */
            for (j = 0; j < tmp_n; ++j)
                buffer[n+j] = local_b[j*n+t];

            for (i = 0; i < local_n; ++i) {
                sum = 0;
                for (j = 0; j < tmp_n; ++j)
                    sum += local_a[i*n+(j*size+new_rank)]*buffer[n+j];
                buffer[i] += sum;
            }

            MPI_Sendrecv_replace(local_b+t, 1, col_t, (rank-1+size)%size, 0,
                                (rank+1)%size, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        }

        sum = 0;
        for (i = 0; i < local_n; ++i)
            sum += fabs(buffer[i]);

        MPI_Reduce(&sum, &col_sum, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
        if (rank == 0 && col_sum > max)
            max = col_sum;
    }

    MPI_Type_free(&col_t);
    if (rank == 0)
        *r = max;
}

void r1_r2_mpi(int n, double *local_a, double *local_x, double *r1, double *r2,
               int rank, int size, double *buffer)
{

```

```

    if (n > 11000) {
        *r1 = 0;
        *r2 = 0;
        return;
    }

    r_mpi(n, local_a, local_x, r1, rank, size, buffer); /*  $||A \cdot A^{(-1)} - E||$  */
    r_mpi(n, local_x, local_a, r2, rank, size, buffer); /*  $||A^{(-1)} \cdot A - E||$  */
}

```

D.2 Задание 1: input.c

```

#include "input.h"

int input_mpi(int s, char *filename, int n, double *local_a, double *buffer, int rank, int size)
{
    if (s == 0)
        return finput_mpi(filename, n, local_a, buffer, rank, size);
    return sinput_mpi(s, n, local_a, rank, size);
}

/* Читает только процесс 0 и сразу отправляет строку её хозяину. */
int finput_mpi(char *filename, int n, double *local_a, double *buffer, int rank, int size)
{
    int i = 0, j = 0, flag = 0;
    FILE *in = NULL;

    if (rank == 0) {
        in = fopen(filename, "r");
        if (in == NULL)
            flag = 1;
    }
    MPI_Bcast(&flag, 1, MPI_INT, 0, MPI_COMM_WORLD);
    if (flag)
        return -1;

    for (i = 0; i < n; ++i) {
        if (rank == 0) {
            for (j = 0; j < n; ++j) {
                if (fscanf(in, "%lf", buffer+j) != 1) {
                    flag = 1;
                    break;
                }
            }
        }
        MPI_Bcast(&flag, 1, MPI_INT, 0, MPI_COMM_WORLD);
        if (flag) {
            if (rank == 0)
                fclose(in);
            return -2;
        }

        if (rank == 0) {
            if (i%size == 0) {
                for (j = 0; j < n; ++j)
                    local_a[(i/size)*n+j] = buffer[j];
            } else {
                MPI_Send(buffer, n, MPI_DOUBLE, i%size, 0, MPI_COMM_WORLD);
            }
        } else if (rank == i%size) {
            MPI_Recv(local_a+(i/size)*n, n, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        }
    }
}

```

```

    if (rank == 0)
        fclose(in);
    return 0;
}

/* Каждый процесс сам считает только свои строки - пересылок нет. */
int sinput_mpi(int s, int n, double *local_a, int rank, int size)
{
    int i = 0, j = 0, local_n = 0;

    if (s < 1 || s > 4)
        return -3;

    local_n = n/size;
    if (rank < n%size)
        ++local_n;

    for (i = 0; i < local_n; ++i)
        for (j = 0; j < n; ++j)
            local_a[i*n+j] = formula_mpi(s, n, i*size+rank, j);

    return 0;
}

/* Формулы из задания. В задании i и j нумеруются с единицы, здесь - с нуля. */
double formula_mpi(int s, int n, int i, int j)
{
    switch (s) {
        case 1:
            return n - (i > j ? i : j);
        case 2:
            return (i > j ? i : j) + 1;
        case 3:
            return (i > j ? i : j) - (i < j ? i : j);
        case 4:
            return 1./(i+j+1);
    }
    return 0;
}

```

D.3 Задание 1: output.c

```

#include "output.h"

void output_mpi(int r, int l, int n, double *local_a, int rank, int size, double *buffer)
{
    int i = 0, j = 0, rows = 0, cols = 0, owner = 0;
    double *row = NULL;

    rows = (l < r ? l : r);
    cols = (n < r ? n : r);

    for (i = 0; i < rows; ++i) {
        owner = i%size;
        row = local_a + (i/size)*n;

        /* строка живёт не на нулевом процессе - пересылаем её туда */
        if (owner != 0) {
            if (rank == owner) {
                MPI_Send(row, n, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD);
            } else if (rank == 0) {
                MPI_Recv(buffer, n, MPI_DOUBLE, owner, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            }
        }
    }
}

```

```

        row = buffer;
    }
}

if (rank == 0) {
    for (j = 0; j < cols; ++j)
        printf(" %10.3e", row[j]);
    printf("\n");
}

if (rank == 0)
    fflush(stdout);
}

```

D.4 Задание 1: main.c

```

#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>
#include "input.h"
#include "output.h"
#include "gauss_reverse_MPI.h"

int main(int argc, char **argv)
{
    int rank = 0, size = 0, n = 0, r = 0, s = 0, task = 14, flag = 0;
    int *col = NULL;
    char *filename = NULL;
    double *local_a = NULL, *local_x = NULL, *buffer = NULL;
    double r1 = 0, r2 = 0, t1 = 0, t2 = 0;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (argc < 4 || argc > 5) {
        if (rank == 0)
            printf("Usage: mpirun -np p %s n r s [filename]\n", argv[0]);
        MPI_Finalize();
        return -1;
    }

    n = strtol(argv[1], NULL, 10);
    r = strtol(argv[2], NULL, 10);
    s = strtol(argv[3], NULL, 10);

    if (n <= 0 || r < 0 || s < 0 || s > 4 || (s == 0 && argc != 5) || (s != 0 && argc != 4)) {
        if (rank == 0)
            printf("Invalid arguments.\n");
        MPI_Finalize();
        return -1;
    }

    if (s == 0)
        filename = argv[4];

    /* 2n^2 + O(n): две матрицы по строкам + буфер 2n + массив перестановки n */
    local_a = (double*)malloc((n/size+1)*n*sizeof(double));
    local_x = (double*)malloc((n/size+1)*n*sizeof(double));
    buffer = (double*)malloc(2*n*sizeof(double));
    col = (int*)malloc(n*sizeof(int));

    if (local_a == NULL || local_x == NULL || buffer == NULL || col == NULL) {

```

```

        if (rank == 0)
            printf("Not enough memory.\n");
        free(local_a); free(local_x); free(buffer); free(col);
        MPI_Finalize();
        return -1;
    }

    flag = input_mpi(s, filename, n, local_a, buffer, rank, size);
    if (flag) {
        if (rank == 0) {
            if (flag == -1)
                printf("Cannot open file %s\n", filename);
            else if (flag == -2)
                printf("Wrong data in file %s\n", filename);
            else
                printf("Wrong formula number: %d\n", s);
        }
        free(local_a); free(local_x); free(buffer); free(col);
        MPI_Finalize();
        return -1;
    }

    output_mpi(r, n, n, local_a, rank, size, buffer);

    MPI_Barrier(MPI_COMM_WORLD);
    t1 = MPI_Wtime();

    flag = gauss_reverse_mpi(n, local_a, local_x, rank, size, buffer, col);

    MPI_Barrier(MPI_COMM_WORLD);
    t1 = MPI_Wtime() - t1;

    if (flag) {
        r1 = -1;
        r2 = -1;
        t2 = 0;
    } else {
        output_mpi(r, n, n, local_x, rank, size, buffer);

        /* памяти на копию A нет, поэтому читаем её заново */
        input_mpi(s, filename, n, local_a, buffer, rank, size);

        MPI_Barrier(MPI_COMM_WORLD);
        t2 = MPI_Wtime();

        r1_r2_mpi(n, local_a, local_x, &r1, &r2, rank, size, buffer);

        MPI_Barrier(MPI_COMM_WORLD);
        t2 = MPI_Wtime() - t2;
    }

    if (rank == 0)
        printf("%s : Task = %d Res1 = %e Res2 = %e T1 = %.2f T2 = %.2f S = %d N = %d\n",
            argv[0], task, r1, r2, t1, t2, s, n);

    free(local_a);
    free(local_x);
    free(buffer);
    free(col);

    MPI_Finalize();
    return 0;
}

```

D.5 Задание 2: newton_interpolation.cpp

```
#include <cmath>
#include <limits>

#include "newton_interpolation.h"

int make_newton_coefficients(int n, const double *x_nodes,
                           const double *f_values, double *coeffs) {
    double dx;

    if (n < 1 || !x_nodes || !f_values || !coeffs) {
        if (coeffs && n >= 1)
            coeffs[0] = std::numeric_limits<double>::quiet_NaN();
        return -1;
    }

    /* Нулевой столбец таблицы разделённых разностей -- значения функции */
    for (int i = 0; i < n; i++)
        coeffs[i] = f_values[i];

    /* Пересчёт "на месте": после j-го прохода coeffs[i] = f[x_{i-j}, ..., x_i] */
    for (int j = 1; j < n; j++) {
        for (int i = n - 1; i >= j; i--) {
            dx = x_nodes[i] - x_nodes[i - j];
            if (std::fabs(dx) < std::numeric_limits<double>::epsilon()) {
                coeffs[0] = std::numeric_limits<double>::quiet_NaN();
                return -1;
            }
            coeffs[i] = (coeffs[i] - coeffs[i - 1]) / dx;
        }
    }

    return 0;
}

double calculate_newton_value(double x_eval, double a, double b, int n,
                             const double *x_nodes, const double *coeffs) {
    double value;

    (void)a;
    (void)b;

    if (n < 1 || !x_nodes || !coeffs || std::isnan(coeffs[0]))
        return std::numeric_limits<double>::quiet_NaN();

    /* Схема Горнера для формы Ньютона:
     *  $P(x) = a_0 + (x - x_1)(a_1 + (x - x_2)(a_2 + \dots))$  */
    value = coeffs[n - 1];
    for (int i = n - 2; i >= 0; i--)
        value = coeffs[i] + (x_eval - x_nodes[i]) * value;

    return value;
}

double calculate_newton_discrepancy(double x_eval, func_t func, double a,
                                    double b, int n, const double *x_nodes,
                                    const double *coeffs) {
    double approx;

    if (!func)
        return std::numeric_limits<double>::quiet_NaN();

    approx = calculate_newton_value(x_eval, a, b, n, x_nodes, coeffs);
    if (std::isnan(approx))
```

```

        return std::numeric_limits<double>::quiet_NaN();

    return func(x_eval) - approx;
}

```

D.6 Задание 2: hermite_natural.cpp

```

#include <algorithm>
#include <cmath>
#include <limits>

#include "hermite_natural.h"

int make_hermite_natural_coefficients(int n, const double *x_nodes,
                                     const double *f_values, double *coeffs,
                                     double *d_values) {
    double h_left, h_right, df_left, df_right, h, df;

    if (n < 2 || !x_nodes || !f_values || !coeffs || !d_values) {
        if (coeffs && n >= 1)
            coeffs[0] = std::numeric_limits<double>::quiet_NaN();
        return -1;
    }

    /* Узлы должны строго возрастать */
    for (int i = 0; i < n - 1; i++) {
        if (x_nodes[i + 1] - x_nodes[i] <= std::numeric_limits<double>::epsilon()) {
            coeffs[0] = std::numeric_limits<double>::quiet_NaN();
            return -1;
        }
    }

    if (n == 2) {
        /* Один отрезок: естественные условия  $S'' = 0$  на обоих концах
         * превращают кубический многочлен в линейный */
        df = (f_values[1] - f_values[0]) / (x_nodes[1] - x_nodes[0]);
        d_values[0] = df;
        d_values[1] = df;
    } else {
        /* Производные во внутренних узлах -- по трём соседним значениям */
        for (int i = 1; i < n - 1; i++) {
            h_left = x_nodes[i] - x_nodes[i - 1];
            h_right = x_nodes[i + 1] - x_nodes[i];
            df_left = (f_values[i] - f_values[i - 1]) / h_left;
            df_right = (f_values[i + 1] - f_values[i]) / h_right;
            d_values[i] = df_left + df_right -
                (f_values[i + 1] - f_values[i - 1]) / (h_left + h_right);
        }

        /* Естественные граничные условия.
         *  $P_{-1}''(x_1) = 0 \Rightarrow 3 f[x_1, x_2] - 2 d_1 - d_2 = 0$ ;
         *  $P_{\{n-1\}}''(x_n) = 0 \Rightarrow 3 f[x_{\{n-1\}}, x_n] - 2 d_n - d_{\{n-1\}} = 0$ . */
        df_left = (f_values[1] - f_values[0]) / (x_nodes[1] - x_nodes[0]);
        d_values[0] = 0.5 * (3. * df_left - d_values[1]);

        df_right = (f_values[n - 1] - f_values[n - 2]) /
            (x_nodes[n - 1] - x_nodes[n - 2]);
        d_values[n - 1] = 0.5 * (3. * df_right - d_values[n - 2]);
    }

    /* Коэффициенты кубических многочленов Эрмита на каждом отрезке */
    for (int i = 0; i < n - 1; i++) {
        h = x_nodes[i + 1] - x_nodes[i];

```

```

        df = (f_values[i + 1] - f_values[i]) / h;

        coeffs[4 * i + 0] = f_values[i];
        coeffs[4 * i + 1] = d_values[i];
        coeffs[4 * i + 2] = (3. * df - 2. * d_values[i] - d_values[i + 1]) / h;
        coeffs[4 * i + 3] = (d_values[i] + d_values[i + 1] - 2. * df) / (h * h);
    }

    return 0;
}

double calculate_hermite_natural_value(double x_eval, double a, double b,
                                       int n, const double *x_nodes,
                                       const double *coeffs) {

    const double *c;
    double t;
    int i;

    (void)a;
    (void)b;

    if (n < 2 || !x_nodes || !coeffs || std::isnan(coeffs[0]))
        return std::numeric_limits<double>::quiet_NaN();

    /* Двоичный поиск отрезка [x_i, x_{i+1}], содержащего точку;
     * вне [x_1, x_n] используется крайний многочлен (экстраполяция) */
    i = static_cast<int>(std::upper_bound(x_nodes, x_nodes + n, x_eval) - x_nodes) - 1;
    if (i < 0)
        i = 0;
    if (i > n - 2)
        i = n - 2;

    t = x_eval - x_nodes[i];
    c = coeffs + 4 * i;
    return c[0] + t * (c[1] + t * (c[2] + t * c[3]));
}

double calculate_hermite_natural_discrepancy(double x_eval, func_t func,
                                             double a, double b, int n,
                                             const double *x_nodes,
                                             const double *coeffs) {

    double approx;

    if (!func)
        return std::numeric_limits<double>::quiet_NaN();

    approx = calculate_hermite_natural_value(x_eval, a, b, n, x_nodes, coeffs);
    if (std::isnan(approx))
        return std::numeric_limits<double>::quiet_NaN();

    return func(x_eval) - approx;
}

```

D.7 Задание 2: parabolic_extrapolation.cpp

```

#include <algorithm>
#include <cmath>
#include <limits>

#include "parabolic_extrapolation.h"

/* Значение в точке t интерполяционного многочлена, построенного по трём
 * узлам (x0, f0), (x1, f1), (x2, f2). Используется для экстраполяции

```



```

 * значения функции в крайние точки склейки, лежащие вне [x_1, x_n]. */
static double extrapolate_by_three_nodes(double t, double x0, double x1,
                                         double x2, double f0, double f1,
                                         double f2) {

    double df01, df12, df012;

    df01 = (f1 - f0) / (x1 - x0);
    df12 = (f2 - f1) / (x2 - x1);
    df012 = (df12 - df01) / (x2 - x0);

    return f0 + (t - x0) * (df01 + (t - x1) * df012);
}

int make_parabolic_extrapolation_coefficients(int n, const double *x_nodes,
                                             const double *f_values,
                                             double *coeffs, double *xi_nodes,
                                             double *v_values) {

    double h_left, z_left, u_left, h_right, z_right, u_right;
    double row_a, row_b, row_c, row_d, denom;
    double h, z, u, p, q, c2;
    double *alpha;
    int size;

    if (n < 2 || !x_nodes || !f_values || !coeffs || !xi_nodes || !v_values) {
        if (coeffs && n >= 1)
            coeffs[0] = std::numeric_limits<double>::quiet_NaN();
        return -1;
    }

    /* Узлы должны строго возрастать */
    for (int i = 0; i < n - 1; i++) {
        if (x_nodes[i + 1] - x_nodes[i] <= std::numeric_limits<double>::epsilon()) {
            coeffs[0] = std::numeric_limits<double>::quiet_NaN();
            return -1;
        }
    }

    /* Точки склейки: середины между узлами плюс два "полушага" наружу */
    xi_nodes[0] = x_nodes[0] - 0.5 * (x_nodes[1] - x_nodes[0]);
    for (int i = 1; i < n; i++)
        xi_nodes[i] = 0.5 * (x_nodes[i - 1] + x_nodes[i]);
    xi_nodes[n] = x_nodes[n - 1] + 0.5 * (x_nodes[n - 1] - x_nodes[n - 2]);

    size = n + 1; /* число неизвестных v_1, ..., v_{n+1} */

    /* Прогонка. Коэффициенты строк вычисляются на ходу, поэтому хранить
     * саму матрицу не нужно; alpha временно размещается в массиве coeffs
     * (в нём 3n >= n+1 элементов), beta -- в массиве v_values. */
    alpha = coeffs;

    /* Строка 0: граничное условие слева -- значение в xi_1 получено
     * экстраполяцией по ближайшим узлам */
    if (n >= 3)
        row_d = extrapolate_by_three_nodes(xi_nodes[0], x_nodes[0], x_nodes[1],
                                           x_nodes[2], f_values[0], f_values[1],
                                           f_values[2]);
    else /* n == 2: экстраполируем линейно по двум узлам */
        row_d = f_values[0] + (xi_nodes[0] - x_nodes[0]) *
                (f_values[1] - f_values[0]) /
                (x_nodes[1] - x_nodes[0]);
    alpha[0] = 0.; /* -c/b при c = 0, b = 1 */
    v_values[0] = row_d; /* beta_0 */

    /* Строки 1..n-1: непрерывность производной сплайна в точках xi_2..xi_n */
    for (int i = 1; i <= n - 1; i++) {

```

```

    h_left = x_nodes[i - 1] - xi_nodes[i - 1];
    z_left = xi_nodes[i] - xi_nodes[i - 1];
    u_left = xi_nodes[i] - x_nodes[i - 1];

    h_right = x_nodes[i] - xi_nodes[i];
    z_right = xi_nodes[i + 1] - xi_nodes[i];
    u_right = xi_nodes[i + 1] - x_nodes[i];

    row_a = z_left / (h_left * u_left) -
        (2. * z_left - h_left) / (z_left * u_left);
    row_b = (2. * z_left - h_left) / (z_left * u_left) +
        z_right / (h_right * u_right) - h_right / (z_right * u_right);
    row_c = h_right / (z_right * u_right);
    row_d = z_left * f_values[i - 1] / (h_left * u_left) +
        z_right * f_values[i] / (h_right * u_right);

    denom = row_b + row_a * alpha[i - 1];
    if (std::fabs(denom) < std::numeric_limits<double>::epsilon()) {
        coeffs[0] = std::numeric_limits<double>::quiet_NaN();
        return -1;
    }
    alpha[i] = -row_c / denom;
    v_values[i] = (row_d - row_a * v_values[i - 1]) / denom;
}

/* Строка n: граничное условие справа -- значение в xi_{n+1} */
if (n >= 3)
    row_d = extrapolate_by_three_nodes(xi_nodes[n], x_nodes[n - 3],
                                       x_nodes[n - 2], x_nodes[n - 1],
                                       f_values[n - 3], f_values[n - 2],
                                       f_values[n - 1]);
else
    row_d = f_values[n - 1] + (xi_nodes[n] - x_nodes[n - 1]) *
        (f_values[n - 1] - f_values[n - 2]) /
        (x_nodes[n - 1] - x_nodes[n - 2]);
/* row_a = 0, row_b = 1 (уравнение v_{n+1} = row_d) */
v_values[n] = row_d;

/* Обратный ход прогонки */
for (int i = size - 2; i >= 0; i--)
    v_values[i] = alpha[i] * v_values[i + 1] + v_values[i];

/* Коэффициенты квадратных трёхчленов на отрезках [xi_i, xi_{i+1}] */
for (int i = 0; i < n; i++) {
    h = x_nodes[i] - xi_nodes[i];
    z = xi_nodes[i + 1] - xi_nodes[i];
    u = xi_nodes[i + 1] - x_nodes[i];

    p = (f_values[i] - v_values[i]) / h;
    q = (v_values[i + 1] - v_values[i]) / z;
    c2 = (q - p) / u;

    coeffs[3 * i + 0] = v_values[i];
    coeffs[3 * i + 1] = p - c2 * h;
    coeffs[3 * i + 2] = c2;
}

return 0;
}

double calculate_parabolic_extrapolation_value(double x_eval, double a,
                                              double b, int n,
                                              const double *x_nodes,
                                              const double *coeffs) {
    const double *c;

```

```

double xi_left, t;
int idx, i;

(void)a;
(void)b;

if (n < 2 || !x_nodes || !coeffs || std::isnan(coeffs[0]))
    return std::numeric_limits<double>::quiet_NaN();

/* Отрезок склейки [xi_i, xi_{i+1}] симметричен относительно узла x_i,
 * поэтому нужный кусок -- тот, чей узел ближе всего к точке */
idx = static_cast<int>(std::upper_bound(x_nodes, x_nodes + n, x_eval) - x_nodes);
if (idx == 0)
    i = 0;
else if (idx == n)
    i = n - 1;
else
    i = (x_eval - x_nodes[idx - 1] <= x_nodes[idx] - x_eval) ? idx - 1 : idx;

xi_left = (i == 0) ? x_nodes[0] - 0.5 * (x_nodes[1] - x_nodes[0])
    : 0.5 * (x_nodes[i - 1] + x_nodes[i]);

t = x_eval - xi_left;
c = coeffs + 3 * i;
return c[0] + t * (c[1] + t * c[2]);
}

double calculate_parabolic_extrapolation_discrepancy(double x_eval,
                                                    func_t func, double a,
                                                    double b, int n,
                                                    const double *x_nodes,
                                                    const double *coeffs) {

    double approx;

    if (!func)
        return std::numeric_limits<double>::quiet_NaN();

    approx = calculate_parabolic_extrapolation_value(x_eval, a, b, n, x_nodes,
                                                    coeffs);

    if (std::isnan(approx))
        return std::numeric_limits<double>::quiet_NaN();

    return func(x_eval) - approx;
}

```

D.8 Задание 3: newton2d.cpp

```

#include <algorithm>
#include <cmath>
#include <limits>
#include <vector>

#include "newton2d.h"

void make_chebyshev_nodes(int n, double a, double b, double *nodes) {
    double center = 0.5 * (a + b);
    double half = 0.5 * (b - a);

    if (n < 1 || !nodes)
        return;

    if (n == 1) {
        nodes[0] = center;
    }
}

```

```

    return;
}

/* cos(...) убывает по i, поэтому заполняем массив с конца,
 * чтобы узлы шли по возрастанию */
for (int i = 0; i < n; i++)
    nodes[n - 1 - i] = center + half * std::cos(M_PI * (i + 0.5) / n);
}

void reorder_nodes_leja(int n, double *nodes) {
    std::vector<double> weight;
    double best_value, center, lo, hi;
    int best;

    if (n < 2 || !nodes)
        return;

    /* Первым берём узел, наиболее удалённый от середины набора узлов */
    lo = hi = nodes[0];
    for (int i = 1; i < n; i++) {
        lo = std::min(lo, nodes[i]);
        hi = std::max(hi, nodes[i]);
    }
    center = 0.5 * (lo + hi);

    weight.assign(n, 0.);
    for (int i = 0; i < n; i++)
        weight[i] = std::fabs(nodes[i] - center);

    for (int k = 0; k < n - 1; k++) {
        best = k;
        best_value = weight[k];
        for (int i = k + 1; i < n; i++) {
            if (weight[i] > best_value) {
                best_value = weight[i];
                best = i;
            }
        }
        std::swap(nodes[k], nodes[best]);
        std::swap(weight[k], weight[best]);

        /* Домножаем веса оставшихся узлов на расстояние до выбранного */
        for (int i = k + 1; i < n; i++)
            weight[i] *= std::fabs(nodes[i] - nodes[k]);
    }
}

int make_newton2d_coefficients(int nx, int ny, const double *x_nodes,
                              const double *y_nodes, const double *f_values,
                              double *coeffs) {
    double dx;

    if (nx < 1 || ny < 1 || !x_nodes || !y_nodes || !f_values || !coeffs) {
        if (coeffs)
            coeffs[0] = std::numeric_limits<double>::quiet_NaN();
        return -1;
    }

    for (int i = 0; i < nx * ny; i++)
        coeffs[i] = f_values[i];

    /* Шаг 1. Разделённые разности по переменной x: для каждого
     * фиксированного j столбец coeffs[*][j] обрабатывается так же,
     * как одномерная таблица разделённых разностей. */
    for (int level = 1; level < nx; level++) {

```

```

    for (int i = nx - 1; i >= level; i--) {
        dx = x_nodes[i] - x_nodes[i - level];
        if (std::fabs(dx) < std::numeric_limits<double>::epsilon()) {
            coeffs[0] = std::numeric_limits<double>::quiet_NaN();
            return -1;
        }
        for (int j = 0; j < ny; j++)
            coeffs[i * ny + j] =
                (coeffs[i * ny + j] - coeffs[(i - 1) * ny + j]) / dx;
    }
}

/* Шаг 2. Разделённые разности по переменной y: теперь то же самое
 * делается внутри каждой строки. Результат -- смешанные разделённые
 * разности  $c_{ij} = f[x_1, \dots, x_i; y_1, \dots, y_j]$ . */
for (int i = 0; i < nx; i++) {
    for (int level = 1; level < ny; level++) {
        for (int j = ny - 1; j >= level; j--) {
            dx = y_nodes[j] - y_nodes[j - level];
            if (std::fabs(dx) < std::numeric_limits<double>::epsilon()) {
                coeffs[0] = std::numeric_limits<double>::quiet_NaN();
                return -1;
            }
            coeffs[i * ny + j] =
                (coeffs[i * ny + j] - coeffs[i * ny + (j - 1)]) / dx;
        }
    }
}

return 0;
}

double calculate_newton2d_value(double px, double py, int nx, int ny,
                                const double *x_nodes, const double *y_nodes,
                                const double *coeffs) {
    double result, inner;

    if (nx < 1 || ny < 1 || !x_nodes || !y_nodes || !coeffs ||
        std::isnan(coeffs[0]))
        return std::numeric_limits<double>::quiet_NaN();

    /* Внешняя схема Горнера по x; коэффициентами служат значения
     * внутренних многочленов по y:
     *  $P(x, y) = g_1(y) + (x - x_1)(g_2(y) + (x - x_2)(\dots))$ ,
     *  $g_i(y) = c_{i1} + (y - y_1)(c_{i2} + (y - y_2)(\dots))$ . */
    result = coeffs[(nx - 1) * ny + ny - 1];
    for (int j = ny - 2; j >= 0; j--)
        result = result * (py - y_nodes[j]) + coeffs[(nx - 1) * ny + j];

    for (int i = nx - 2; i >= 0; i--) {
        inner = coeffs[i * ny + ny - 1];
        for (int j = ny - 2; j >= 0; j--)
            inner = inner * (py - y_nodes[j]) + coeffs[i * ny + j];

        result = result * (px - x_nodes[i]) + inner;
    }

    return result;
}

double calculate_newton2d_discrepancy(double px, double py, func2_t func,
                                        int nx, int ny, const double *x_nodes,
                                        const double *y_nodes,
                                        const double *coeffs) {
    double approx;

```

```
if (!func)
    return std::numeric_limits<double>::quiet_NaN();

approx = calculate_newton2d_value(px, py, nx, ny, x_nodes, y_nodes, coeffs);
if (std::isnan(approx))
    return std::numeric_limits<double>::quiet_NaN();

return func(px, py) - approx;
}
```